

Güvenli Programlamaya Giriş ve Uygulama Koruma Planı

CEN429 Güvenli Programlama — Hafta 1

Dr. Öğr. Üyesi Uğur CORUH · 18.09.2026

Bugünün planı (3 saat)

Saat	Konu
1	Ders tanıtımı · Güvenlik nedir? · Saldırgan · İlkeler · Uygulama korumasına genel bakış
2	Koruma planı · STRIDE ve risk puanı · Saldırı ağacı · Uygulamalı örnek "Kasa" · Sınıf alıştırmaları
3	Demo 1–2 · Güvenli başlatma · Taşmalar · Demo 3–4 · Bellek yönetimi · Bölümleme · Güvenli süreç · Proje

Öğrenme çıktıları: ÖÇ.1 (zafiyetleri tanır ve sınıflandırır) · ÖÇ.5 (güvenli tasarım ilkeleriyle yazılım planı oluşturur)

Bu derse nasıl başlıyoruz

- Bu, dersin **ilk haftası**; önceki bir haftaya dayanmıyoruz.
- Varsayılan ön bilgi: C'de fonksiyon, dizi, işaretçi; terminalde `cd`, `ls` (PowerShell ya da Linux).
- Aşağıdaki "**Bu haftanın kavramları**" tablosu, bu haftanın terimlerini birer cümleyle tanımlamak yerine, her birinin **hangi bölümde** öğretileceğini gösterir.
- Dönem boyunca izleyeceğimiz kalıp: **hatalı kod** → **saldırı** → **düzeltilme**.

Bu haftanın kavramları

Her terim, gövdede ilk geçtiği yerde tanımlanır; burada yalnız **nerede** olduğunu işaretliyoruz.

Kavram	Nerede
Güvenlik	Bölüm 1
Varlık (asset)	Bölüm 1
Tehdit ve zafiyet	Bölüm 1
CIA üçlüsü	Bölüm 1
Saldırgan modeli	Bölüm 1
Beyaz kutu / MATE	Bölüm 1
Güvenli tasarım ilkeleri	Bölüm 1
Katmanlı savunma	Bölüm 2
Ödünleşim (trade-off)	Bölüm 2
Tehdit modelleme	Bölüm 3
STRIDE	Bölüm 3
Saldırı ağacı	Bölüm 3
Veri akış diyagramı (DFD)	Bölüm 3

Demolar nasıl çalışıyor?

- Tek kaynak, iki/üç ortam: **Windows** (Visual Studio 2022 Community ya da PowerShell), **WSL** ve **Linux**
- Derle: Windows `.\build.ps1` · WSL/Linux `./build.sh` — dersten önce bir kez derleyin
- Çalıştır: Windows `.\demo.ps1` · WSL/Linux `sh demo.sh`
- Visual Studio: **Dosya > Aç > Klasör** → `code` → yapılandırma **Windows (MSVC)** → **Derle > Tümünü Derle**
- Bu hafta 4 demo: `code/week-01` altında

 **Etik:** Bütün demolar yalnız kendi klasöründe çalışır, yönetici yetkisi istemez. Teknikleri yalnız **kendi bilgisayarınızda** deneyin.

Kısa tarihçe — güvenli programlama fikri

- **1975** — Saltzer & Schroeder: güvenli tasarımın **8 ilkesi** (en az ayrıcalık, derinlemesine savunma...)
- **1970–80'ler** — **CIA üçlüsü** ortak dil hâline gelir
- **1998–99** — Microsoft'ta **STRIDE**; Schneier **saldırı ağaçlarını** tanıtır
- **2001–03** — *Building Secure Software* ve **Secure Programming Cookbook** (dersin ana kaynağı)

Bugünkü araçlar (CIA · saldırgan modeli · STRIDE · saldırı ağacı) bu çizginin ürünüdür.

Güvenli programlamanın beş kararı — şema

Neden “güvenli programlama”? Beş karar, kodun içinde

güvenlik duvarı hatalı yazılmış programı kurtaramaz

Girdiye güvenme

uzunluk, tür, aralık, izin verilen karakterler — her seferinde

Sırrı kısa tut

işin bitince güvenle sil; bellekte bekletme

Ortama güvenme

ortam değişkenleri, dosya izinleri, çağrılan programlar

Güvenli tarafta kal

hata olursa reddet, kapat, sızdırma

Korumaları aç ve sına

derleyici bayrakları, sanitizer, fuzzing

Saldırgan izin verilen kapıdan girer: programın kendisinden.

Ders nasıl yürüyor?

- **Tek dönem projesi:** C/C++ uygulaması + "sertifikasyondan geçecekmiş gibi" bir **güvenlik kılavuzu**
 - Vize kontrolü: **7. hafta** gösterim · Final kontrolü: **15. hafta** gösterim
- **İki quiz:** 8. hafta (1–6) · 16. hafta (9–14)
- Not: $\text{Vize} = 0,6 \cdot \text{Proje1} + 0,4 \cdot \text{Quiz1}$ · $\text{Final} = 0,7 \cdot \text{Proje2} + 0,3 \cdot \text{Quiz2}$
- Her hafta: **hatalı kod** → **saldırı** → **düzeltilme**



Etik: Teknikleri yalnız **kendi bilgisayarınızda** ve verilen demolar üzerinde deneyin.

1. Güvenlik nedir?

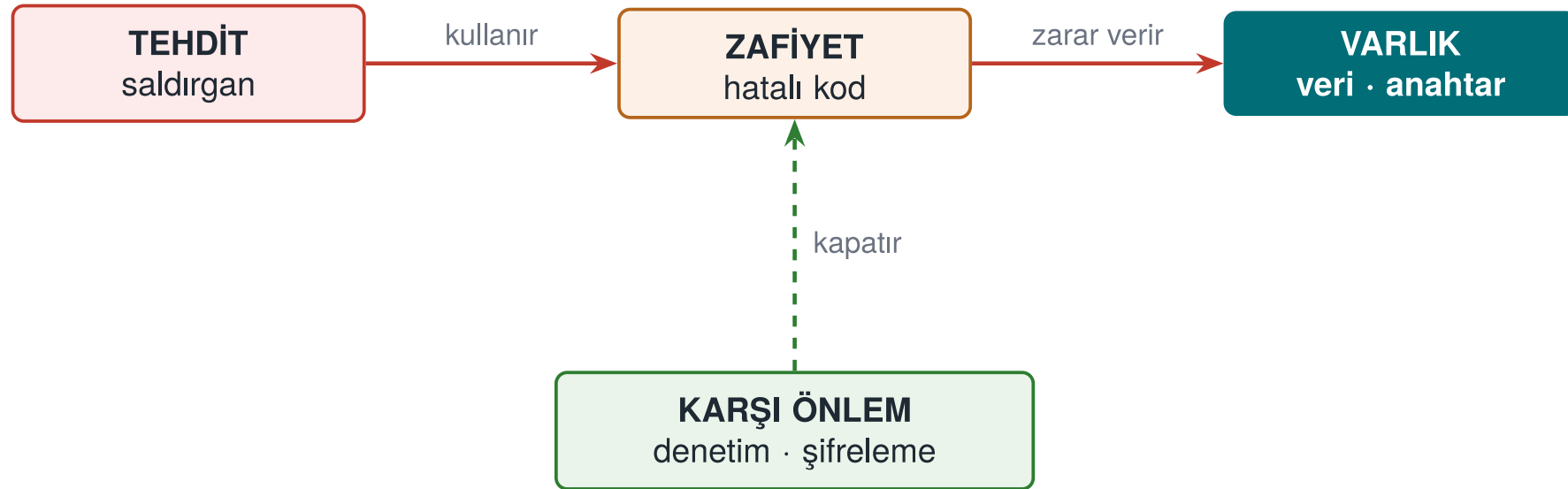
Kasa benzetmesi

Kavram	Kasa	Yazılım
Varlık	Para	Parola, ödeme anahtarı
Tehdit	Hırsız	Parolayı çalmak isteyen
Zafiyet	Arızalı kilit	Denetlenmeyen <code>strcpy</code>
Saldırı	Kilidi açmak	Uzun adla yönetici olmak
Karşı önlem	Yeni kilit	Uzunluk denetimi
Risk	Olasılık $\times$ etki	"Bulunursa bütün hesaplar gider"

Tehdit · zafiyet · varlık — şema

Tehdit · zafiyet · varlık · karşı önlem

risk = olasılık × etki



Zafiyet olmadan tehdit zarar veremez; karşı önlem zafiyeti kapatır.

Güvenliğin üç hedefi: CIA

	Soru	Bozulursa
C onfidentiality — Gizlilik	Yalnız yetkili görebiliyor mu?	Parolalar sızar
I ntegrity — Bütünlük	Yetkisiz değiştirilebiliyor mu?	Bakiye, not değişir
A vailability — Erişilebilirlik	Gerektiğinde çalışıyor mu?	Sistem çöker

+ Kimlik doğrulama · Yetkilendirme · İnkâr edilemezlik

Soru: Bankacılık uygulaması bakiyeyi şifresiz yazıyor → ? · Alıcı IBAN yolda değişiyor → ? · Bayram gecesi çöküyor → ?

CIA üçlüsü — şema

CIA üçlüsü

her güvenlik hedefi bu üçünden birine dayanır

GİZLİLİK
yalnız yetkili
GÖREBİLİR

parola sızarsa
ihlal

BÜTÜNLÜK
yalnız yetkili
DEĞİŞTİREBİLİR

bakiye değişirse
ihlal

ERİŞİLEBİLİRLİK
gerektiğinde
ÇALIŞIR

hizmet durursa
ihlal

Bir saldırıyı sınıflarken önce sorun: bu üçünden HANGİSİ bozuldu?

Sorunun cevabı: bakiye şifresiz → **C**; IBAN yolda değişiyor → **I**; bayram gecesi çöküyor → **A**.

CIA üçlüsü — tipik önlemler ve komşu kavramlar

Hedef	Tipik önlem
Gizlilik	Şifreleme, erişim denetimi
Bütünlük	Özet (hash), MAC, dijital imza
Erişilebilirlik	Yedeklilik, kaynak sınırlama

+ **Kimlik doğrulama:** sen gerçekten o musun? · **Yetkilendirme:** bunu yapmaya iznin var mı? · **İnkâr edilemezlik:** bunu kim yaptı, sonradan inkâr edebilir mi?

Hata ≠ zafiyet, ama...

Güvenlik açıklarının çok büyük kısmı **sıradan yazılım hatalarıdır**.

Heartbleed (2014, CVE-2014-0160)

- İstemci: "Bana **64 KB** geri yolla" — ama yalnız **birkaç bayt** gönderir
- Sunucu uzunluğu **denetlemez** → komşu belleği (parolalar, anahtarlar) geri yollar
- Tek bir eksik uzunluk denetimi → internetin önemli bir kısmı etkilendi

Saldırgan kim, neye erişebiliyor?

Saldırgan	Erişim	Savunma
Uzaktaki	Yalnız ağ mesajları	Girdi doğrulama, TLS
Yerel kullanıcı	Dosyalar, ortam değişkenleri	İzinler, güvenli başlatma
İçeriden kişi	Kaynak kod, sunucu	Ayrıcalık ayrımı, denetim kaydı
Cihazın sahibi (beyaz kutu)	Bellek, hata ayıklayıcı, binary kod	RASP, kod gizleme, whitebox

Bu dersin farkı: program **saldırganın cihazında** çalışıyorsa ne yaparız?

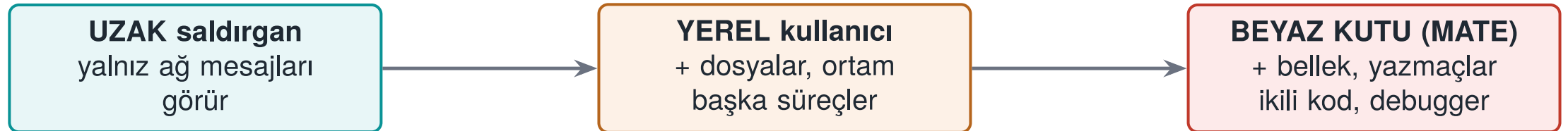
Beyaz kutu / MATE

- **MATE (Man-At-The-End):** programa **sahip** saldırgan.
- Kodu okur, belleği görür, değiştirir.
- Mobil/masaüstü uygulamanın gerçek durumu (bu dersin ana teması).

Saldırgan modelleri — şema

Saldırgan modelleri: ne görebiliyor?

soldan sağa görünürlük artar → savunma zorlaşır



Bu derste varsayılan model BEYAZ KUTU'dur: saldırı cihazın sahibidir.

Saldırganlar — gerçek hayattan örnek

Saldırgan	Örnek
Uzaktaki	Web sunucusuna saldıran biri
Yerel kullanıcı	Paylaşılan sunucudaki başka bir öğrenci
İçeriden kişi	Görevini kötüye kullanan çalışan
Cihazın sahibi (beyaz kutu)	Mobil ödeme uygulamasını kendi telefonunda kırmaya çalışan biri

Her satırı kendi projenize uygulayın: sizin saldırganınız hangisi?

Güvenli tasarım ilkeleri (Saltzer & Schroeder, 1975)

İlke	Tek cümle
En az ayrıcalık	Yalnız gereken yetkiyle çalış
Güvenli varsayılan	Varsayılan "izin yok"
Tam aracılık	Her erişimi her seferinde denetle
Açık tasarım	Güvenlik algoritmanın değil anahtarın gizliliğine dayanır
Mekanizma ekonomisi	Basit = daha az hata
Ayrıcalıkların ayrılması	Kritik işe iki koşul
En az ortak mekanizma	Paylaşılan kaynağı azalt
Kabul edilebilirlik	Kullanılamayan güvenlik atlatılır

+ **Derinlemesine savunma**: tek önleme güvenme, **katman** kur.

Saltzer ve Schroeder ilkeleri — şema

Saltzer ve Schroeder (1975): güvenli tasarımın omurgası

elli yıl sonra hâlâ geçerli sekiz ilke

En az ayrıcalık
işini yapacak kadar yetki

Güvenli varsayılan
varsayılan: izin YOK

Tam aracılık
her erişim her seferinde denetlenir

Açık tasarım
gizlilik anahtarda, tasarımda değil

Mekanizma ekonomisi
basit tasarım = az hata

Ortak mekanizmayı azalt
paylaşılan yol = paylaşılan risk

Yetki ayrımı
tek onay yetmez

Kullanılabilirlik
zor gelen güvenlik atlanır

Bu ilkeler bir denetim listesi değil, tasarım kararlarını tartarken kullanılan ölçülerdir.

Güvenli tasarım ilkeleri — örnekler (1/2)

İlke	Örnek
En az ayrıcalık	Rapor programı yönetici değil, sıradan kullanıcıyla çalışır
Güvenli varsayılan	Yeni kullanıcının admin alanı 0 ile başlar
Tam aracılık	Yetki bir kez bakılıp önbelleğe yazılmaz
Açık tasarım	Kendi şifreleme algoritmanızı yazmazsınız

Her ilke soyut değil; kodda somut bir kararla karşılık bulur.

Güvenli tasarım ilkeleri — örnekler (2/2)

İlke	Örnek
Mekanizma ekonomisi	20 satırlık açık denetim, 500 satırlık "akıllı" denetimden iyidir
Ayrıcalıkların ayrılması	Para transferi için hem parola hem SMS kodu
En az ortak mekanizma	Her oturumun kendi geçici dosyası olur
Psikolojik kabul edilebilirlik	Her 5 dakikada parola sormak, parolanın kâğıda yazılmasına yol açar

Kullanılamayan güvenlik, **atlatılan** güvenlidir.

2. Uygulama korumasına genel bakış

Katmanlı savunma

- **Katmanlı savunma (defense in depth):** tek önlem değil, **birçok** önlem.
- Biri aşılsa diğeri durdurur.
- "Güvenlik kabuğu" bu katmanların birleşimi.

Neden tek bir önlem yetmez?

Önlem	Neyi çözmez?
Hatasız kod	Saldırgan binary dosyayı açıp mantığı okur
Gizlenmiş kod	Hata ayıklayıcıyla durdurup belleği okur
İzlenmeye karşı korunan bellek	Anahtar kodda düz metin → dize taraması yeter
Gizlenmiş anahtar	Tek bir denetimi değiştirip korumayı atlar

→ Her katman bir öncekinin açığını kapatır; saldırgan hepsini **aynı anda** aşmalı

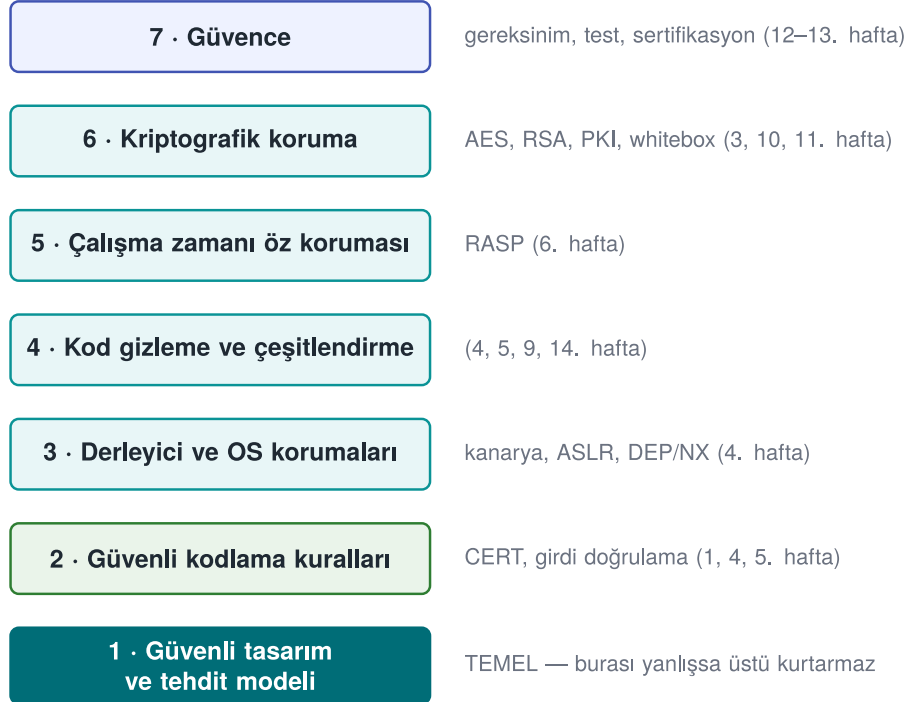
Yedi katman: içten dışa

#	Katman	Soru	Hafta
1	Güvenli tasarım	Neyi, kime karşı?	1–2
2	Güvenli kodlama	Kodum hatasız mı?	1, 4, 5
3	Derleyici/OS korumaları	Hata kaçarsa sömürü zor mu?	4
4	Gizleme	Kodu okuyan anlayabiliyor mu?	4, 5, 9, 14
5	RASP	Saldırı altında olduğunu fark ediyor mu?	6
6	Kriptografi	Veri ele geçse bile anlamsız mı?	3, 10, 11
7	Güvence	Çalıştığını nasıl kanıtlıyoruz?	12–13

Yedi katman — şema (içten dışa)

Yedi koruma katmanı

aşağıdan yukarı: her katman bir öncekinin üzerine kurulur



Bir katman aşılsa bile diğerleri durdurur — derinlemesine savunma.

Bir hata dış katmanları geçse bile **bir sonraki** katman durdurmayı dener (derinlemesine savunma).

Yedi katman — tipik teknikler ve durdurduğu saldırgan

#	Katman	Tipik teknikler	Durduğu saldırgan
1	Tasarım	Tehdit modeli, varlık/arayüz tabloları, en az ayrıcalık	Tasarım hatası arayan herkes
2	Kodlama	Girdi doğrulama, sınır denetimi, SEI CERT kuralları	Uzaktan girdi gönderen saldırgan
3	Derleyici/OS	Kanarya, ASLR, DEP/NX, CFI, <code>_FORTIFY_SOURCE</code>	Bellek hatasını kullanan saldırgan
4	Gizleme	Kontrol akışı düzleştirme, opak yüklem, sanallaştırma	Tersine mühendislik yapan analist
5	RASP	Hata ayıklayıcı/kanca/root algılama, bütünlük denetimi	Programı çalışırken inceleyen saldırgan
6	Kriptografi	Kimlik doğrulamalı şifreleme, anahtar hiyerarşisi, whitebox	Veriyi ele geçiren saldırgan
7	Güvence	Kod incelemesi, fuzzing, sızma testi, sertifikasyon	Denetçi ve değerlendirici

Güvenlik kabuğu: katmanların birleştiği yer

Hassas bir anahtar **aynı anda** birkaç katmanın içinde taşınır:

1. Diskte **şifreli** durur
2. Yalnız kullanılacağı an, yalnız gerektiği kadar **çözülür**
3. Çözüldüğü kod bölümü **gizlenmiş** ve **bütünlüğü denetlenmiş**
4. Süreçte **hata ayıklayıcı denetimi** çalışıyor

➔ 3. haftada dört kabuklu bir tasarımı adım adım kuracağız

Hangi katman ne zaman? Saldırgan modeline göre

Uygulama	Saldırgan nerede?	Öncelik
Sunucu web hizmeti	Ağda; kod ve sunucuya erişimi yok	1, 2, 3, 6, 7
Masaüstü iş uygulaması	Kullanıcı bilgisayarında; kullanıcı saldırgan değil	1, 2, 3, 6
Mobil ödeme	Cihaz saldırganın elinde olabilir	Hepsi; özellikle 4, 5, 6
Oyun, lisanslı yazılım	Kullanıcı saldırgan	4, 5
Gömülü cihaz	Fiziksel erişim	2, 3, 6 + donanım

Beyaz kutu saldırgan: programın kendisine sahip — kopyalar, açar, değiştirir, tekrar tekrar çalıştırır



Gizleme doğru kodun yerine geçmez

- Gizleme saldırganı **yavaşlatır**, hatayı **düzeltilmez**
- Gizlenmiş bir taşma hâlâ bir taşmadır
- Dış katmanlar ancak iç katmanlar sağlamsa anlam kazanır

Sunucu, istemciye asla güvenmez. Gizlenmiş istemci bile sunucuya tutarsız veri gönderebilir.

Beyaz kutu saldırganın altı yolu (1/2)

İstemci tarafında çalışan hassas bir kütüphane için altı satırlık bir **saldırı tablosu**:

#	Saldırganın yolu	Örnek	Cevap veren katman
1	Platform denetimlerini aşmak	Root'lu cihaz, emülatör, değiştirilmiş OS	5 (RASP)
2	Kodu tersine mühendislikle çözmek	Tersine derleyiciyle mantık/anahtar okumak	4 (gizleme), 6
3	Kodu değiştirmek	Bir <code>if</code> denetimini ters çevirip yeniden paketlemek	5 (bütünlük), 4

Beyaz kutu saldırganın altı yolu (2/2)

#	Saldırganın yolu	Örnek	Cevap veren katman
4	Arayüzleri kötüye kullanmak	Kütüphaneyi kendi programından çağırmak	1 (tasarım), 5, 6
5	Çalışma anında varlık çıkarmak	Bellek dökümü, hata ayıklayıcı, kanca	2 (silme), 5, 6 (whitebox)
6	Çalışma anında akışı değiştirmek	Bir dalı atlamak, dönüş değerini değiştirmek	5 (akış sayacı), 4

Bu tablo, koruma planındaki **tehdit** bölümünün başlangıç noktasıdır.

Ödünleşim (trade-off)

- Her koruma bir **bedel** getirir: hız, karmaşıklık, maliyet.
- Güvenlik "sonsuz koruma" değil, **dengeli** koruma.
- Kalan riski açıkça yazarız.

Korumanın bedeli

Bedel	Örnek
Performans	Kontrol akışı düzeltirme bir fonksiyonu birkaç kat yavaşlatabilir
Boyut	Sanallaştırma, whitebox tabloları binary dosyayı büyütür
Bakım	Gizlenmiş kodun çökme raporu okunmaz
Yanlış pozitif	Aşırı hassas root algılama meşru kullanıcıyı cezalandırır

→ Her önlem **gerekçesiyle** yazılır: *şu varlığı, şu saldırıgana karşı, şu maliyetle*

Değerlendirici tabloyu tersten okur

1. Dize taraması → anahtar, URL, hata iletisi görünüyor mu?
2. Binary dosyayı açma → mantık okunuyor mu?
3. Hata ayıklayıcı bağlama → program fark ediyor mu?
4. Kodu değiştirme → bütünlük denetimi yakalıyor mu?
5. Bellek dökümü → sır bulunuyor mu?

Durdurulmadığı **her adım** raporda bir bulgu olur

Kendini dene (3 dk)

1. **Çevrimdışı parola yöneticisi** için saldırgan modeli nedir? Hangi katmanlar önemli?
2. "Sunucuda çalışıyor, gizlemeye gerek yok" — ne zaman doğru, ne zaman yanlış?
3. Yalnız gizleme kullanan bir oyunda hangi yol açık kalır?

Kendini dene — cevaplar

1. **Çevrimdışı parola yöneticisi**: saldırgan cihaza/dosyaya sahip (beyaz kutu). Önemli katmanlar: **kriptografi** (paroladan KDF + AEAD), **güvenli kodlama**, bir miktar gizleme/RASP.
2. **"Sunucuda çalışıyor"**: istemciye hassas mantık/anahtar **gönderilmiyorsa doğru**; istemcide çalışan kod/anahtar varsa **yanlış** (beyaz kutu).
3. **Yalnız gizleme**: sunucu tarafı doğrulama yoksa, istemci hilesi (yamalı istemci) açık kalır — gizleme mantığı **düzelmez**.

3. Uygulama koruma planı ve tehdit modelleme

Tehdit modelleme

- **Tehdit modelleme:** "neyi, kime karşı, nasıl koruyacağız?" sorusunu **sistemli** yanıtlamak.
- Varlıkları, tehditleri, önlemleri listeler.
- Bugün STRIDE ve saldırı ağaçlarıyla yapacağız.

Uygulama koruma planı: 7 adım

1. **Kapsam** — Neyi koruyoruz? Hangi binary, hangi sürüm, hangi özet?
2. **Mimari ve arayüzler** — Kim kiminle, hangi kanaldan konuşuyor?
3. **Varlıklar** — Nerede durur, ne zaman silinir, **C** mi **I** mi?
4. **Tehditler** — Saldırgan kim, varlığa hangi yoldan ulaşır?
5. **Karşı önlemler** — Hangi katman, hangi önlem?
6. **Doğrulama** — Çalıştığı nasıl kanıtlanıyor?
7. **Kalan risk** — Kabul edilenler ve nedenleri

→ Sertifikasyonda bu plan **yüzlerce sayfalık bir güvenlik kılavuzudur**. Siz projede **20–30 sayfalık** sürümünü yazacaksınız.

Adım 1 detay: Kapsam

Soru: Neyi değerlendiriyoruz?

- Yalnız "sürüm 2.1" demek yetmez
- Hangi **binary dosya**, hangi **kaynak kod**, hangi **özet (hash)** değeri?
- Sertifikasyonda buna **değerlendirme hedefi** denir

Adım 2 detay: Mimari ve arayüzler

Soru: Bileşenler neler, nasıl konuşuyor?

- Bileşenler **nelerdir**?
- Birbirleriyle **hangi kanaldan** konuşuyorlar?
- Her kanalda kimlik **nasıl doğrulanıyor**?

Adım 3 detay: Varlıklar

Soru: Korunacak veri nerede, ne zaman?

- Korunacak her veri **tek tek** listelenir
- **Nerede** durur, **ne zaman** oluşur, **ne zaman** silinir?
- **Gizlilik (C)** mi, **bütünlük (I)** mü gerekir?

Adım 4 detay: Tehditler

Soru: Saldırgan kim, nasıl ulaşır?

- Saldırgan **kim**? (bkz. saldırgan modelleri)
- Her varlığa **hangi yoldan** ulaşabilir?
- STRIDE ve saldırı ağacı burada devreye girer

Adım 5 detay: Karşı önlemler

Soru: Hangi önlem, hangi katmanda?

- Her tehdit için **bir önlem** seçilir
- Önlem, yedi katmandan **hangisine** ait?
- Gerekçesiz önlem = **yanlış yerde** ya da **gereksiz**

Adım 6 detay: Doğrulama

Soru: Çalıştığı nasıl kanıtlanıyor?

- Her önlem için bir **test** yazılır
- Birim testi mi, **atlatma denemesi** mi?
- Testi olmayan önlem, değerlendirici gözünde **yoktur**

Adım 7 detay: Kalan risk

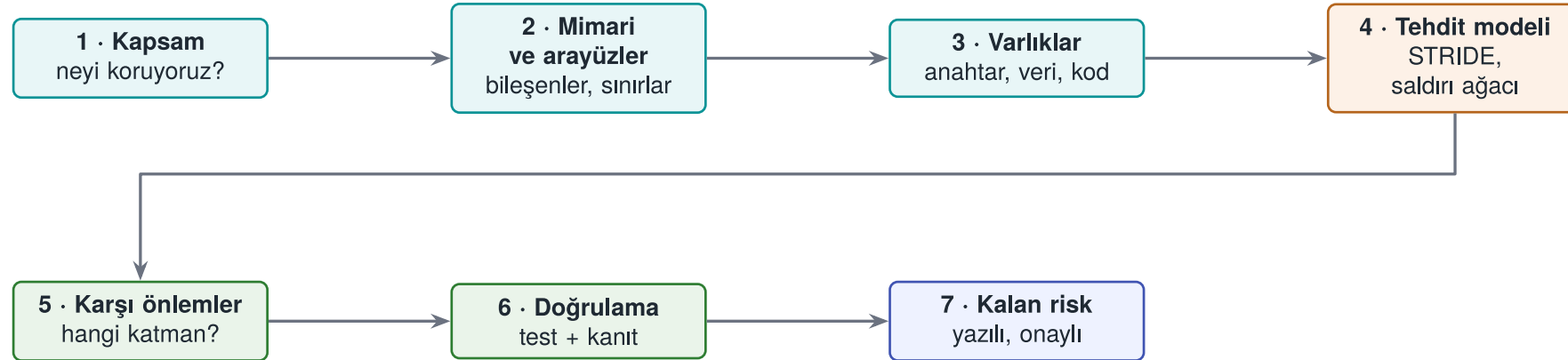
Soru: Neyi bilerek kabul ettik?

- Kapatılamayan ya da bilerek **kabul edilen** riskler
- Her biri için **neden** (ör. performans, kullanılabilirlik)
- Plan **bitmez**: ürün değiştikçe adım 1'e dönülür ve **güncellenir**

Koruma planı — şema

Uygulama koruma planı: yedi adım

1–3 TANIMLA · 4–7 KORU ve KANITLA



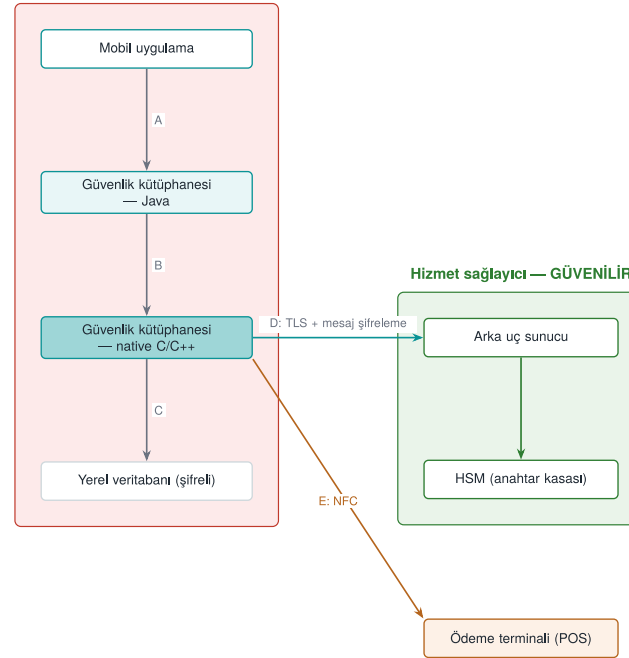
Bu yedi adım dönem projesinin de iskeletidir (S2–S17).

Örnek mimari: mobil ödeme uygulaması

Mobil ödeme: mimari ve arayüzler

kırmızı kutu = güvenli ortam (saldırgan burada)

Kullanıcının telefonu — GÜVENİLMEZ



Her arayüz (A–E) ayrı bir saldırı yüzeyidir: her biri için önlem ve kanıt yazılır.

- Telefon **saldırganın elinde** olabilir: root, hata ayıklayıcı, bellek dökümü
- En hassas işler **native C/C++** katmanında

Arayüz ve varlık tabloları

Arayüz	Uçlar	Kimlik doğrulama	Gizlilik / bütünlük
C	native – veritabanı	—	Şifreleme + MAC, anahtar cihaza bağlı
D	SDK – sunucu	Sertifika sabitleme	TLS + ayrıca mesaj şifreleme

Varlık	Nerede	Ömür	Koruma
Tek kullanımlık ödeme anahtarı	DB (şifreli), bellek	Tek ödeme	C, I
Oturum anahtarı	Yalnız bellek	Oturum	C, I
İşlem sayacı	DB	Kalıcı	I

Listede olmayan varlık korunmuyor demektir.

Arayüz tablosu — tam liste (mobil ödeme)

Arayüz	Uçlar	Kimlik doğrulama	Gizlilik / bütünlük
A	Uygulama – Kütüphane (Java)	Çağıranın imzası doğrulanır	Aynı süreç içi; hassas veri geçmez
B	Kütüphane (Java) – native	Karşılıklı bütünlük denetimi	Şifreli ve örtülü geçer
C	native – yerel veritabanı	—	Şifreleme + MAC, anahtar cihaza bağlı
D	Kütüphane – arka uç sunucu	Sertifika sabitleme + sunucu doğrulama	TLS + ayrıca mesaj düzeyi şifreleme
E	Kütüphane – ödeme terminali	Ödeme protokolü	Tek kullanımlık ödeme anahtarı

Varlık tablosu — tam liste (mobil ödeme)

Varlık	Nerede	Oluşma → silinme	Koruma
Tek kullanımlık ödeme anahtarı	DB (şifreli), bellek	Sunucudan iner → kullanılır → silinir	C, I
Cihaz parmak izi	Bellek, iki ayrı parça	Açılışta hesaplanır	I
Oturum anahtarı	Yalnız bellek	Türetilir → oturum sonunda silinir	C, I
İşlem sayacı	Veritabanı	Her ödemede artar	I
Sunucu açık anahtarı	Uygulamaya gömülü	Derleme anında	I

STRIDE

	Tehdit	Bozduğu	Örnek
S	Kimlik taklidi	Kimlik doğrulama	Sahte uygulama SDK'yı çağırır
T	Kurcalama	Bütünlük	Native denetim binary dosyada değiştirilir
R	İnkâr	İnkâr edilemezlik	"Bu ödemeyi ben yapmadım"
I	Bilgi sızması	Gizlilik	Anahtar bellek dökümünden okunur
D	Hizmet engelleme	Erişilebilirlik	Binlerce sahte kayıt isteği
E	Yetki yükseltme	Yetkilendirme	Sıradan kullanıcı → yönetici

Veri akış diyagramını çiz → **güven sınırını geçen her ok** için altı harfi sor.

Örnek sistem

Bir mobil bankacılık uygulaması:

- kullanıcı girişi
- bakiye görüntüleme
- para transferi

Her STRIDE harfini bu sisteme uygulayalım.

S · Spoofing (sahtecilik)

- **Tehdit:** başkası gibi davranmak (sahte kullanıcı/sunucu).
- **Örnek:** sahte sunucu, çalınan oturum.
- **Önlem:** güçlü kimlik doğrulama, TLS + sertifika denetimi.

T · Tampering (kurcalama)

- **Tehdit:** veriyi/kodu izinsiz değiştirmek.
- **Örnek:** transfer tutarını değiştirmek.
- **Önlem:** bütünlük (MAC/imza), bütünlük denetimi (RASP).

R · Repudiation (inkâr)

- **Tehdit:** yaptığıııı inkâr etmek.
- **Örnek:** "ben o transferi yapmadım".
- **Önlem:** güvenli günlük, imza, denetim izi.

I · Information disclosure (bilgi sızması)

- **Tehdit:** gizli verinin sızması.
- **Örnek:** bakiye/parola sızıntısı.
- **Önlem:** şifreleme, en az yetki, sızıntısız hata iletisi.

D · Denial of service (hizmet reddi)

- **Tehdit:** hizmeti kullanılamaz kılmak.
- **Örnek:** aşırı istek, kaynak tüketimi.
- **Önlem:** hız sınırı, kaynak kotaları, girdi sınırı.

E · Elevation of privilege (yetki yükseltme)

- **Tehdit:** izinsiz yetki kazanmak.
- **Örnek:** normal kullanıcı yönetici olur.
- **Önlem:** en az yetki, yetki denetimi, güvenli varsayılan.

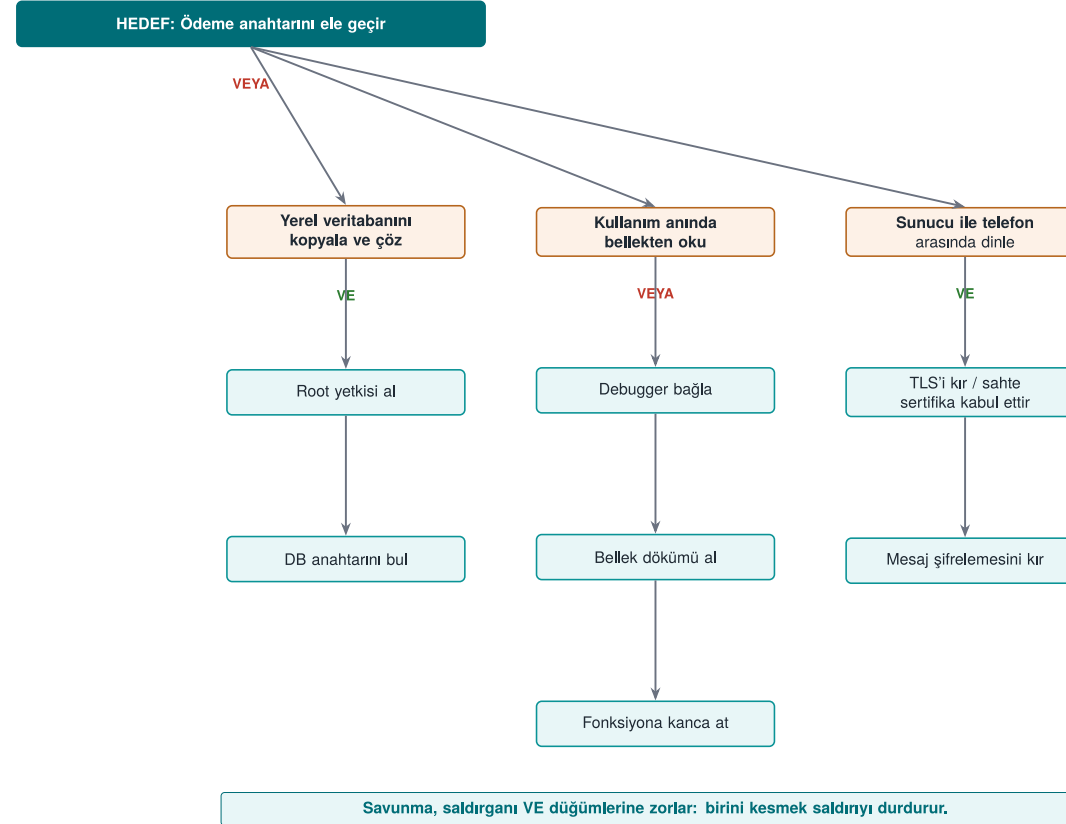
Bankacılık örneğinde STRIDE özeti

Harf	Bu sistemde önlem
S	TLS + kimlik doğrulama
T	MAC/imza + RASP
R	Güvenli günlük
I	Şifreleme + en az yetki
D	Hız sınırı
E	Yetki denetimi

Kök · hedef

Saldırı ağacı: Ödeme anahtarını ele geçir

VEYA = tek dal yeter · VE = tüm dallar gerekir



Saldırganın nihai amacı. Şimdi yolları bölelim.

Saldırı ağacı: "Ödeme anahtarını ele geçir"

HEDEF: Ödeme anahtarı — *VEYA* (biri yeter)

1. **Veritabanını çöz** — *VE* (hepsi gerekir): root yetkisi al + veritabanı anahtarını bul
2. **Bellekten oku** — *VEYA*: hata ayıklayıcı bağla | bellek dökümü al | fonksiyona kanca at
3. **Trafiği dinle** — *VE*: TLS'i kır + mesaj düzeyi şifrelemeyi kır

Saldırı ağacı — "Veritabanını çöz" dalı

VE düğümü (hepsi gerekir):

1. Root yetkisi al
2. **VE** veritabanı anahtarını bul

İkisi birden gerektiği için bu yol **pahalı**dır.

Bu, derinlemesine savunmanın **çalıştığı**nın kanıtıdır.

Saldırı ağacı — "Bellekten oku" dalı

VEYA düğümü (biri yeter): üç alternatif

1. Hata ayıklayıcı bağla
2. **VEYA** bellek dökümü al
3. **VEYA** fonksiyona kanca at

Üç alternatif = **en zayıf nokta**: bellekteki anahtar.

→ Bu hafta Demo 2 · 6. hafta çalışma zamanı koruması · 11. hafta whitebox

Saldırı ağacı — "Trafiği dinle" dalı

VE düğümü (hepsi gerekir):

1. TLS'i kır
2. **VE** mesaj düzeyi şifrelemeyi kır

İki katmanı **birden** kırmak gerekir → pahalı yol.

Savunmacının hedefi: saldırganı mümkün olduğunca çok **VE** düğümüne zorlamak.

Saldırı ağacı — önlem eşlemesi

Saldırı ağacının yapraklarını doğrudan bir savunma katmanına bağlayalım:

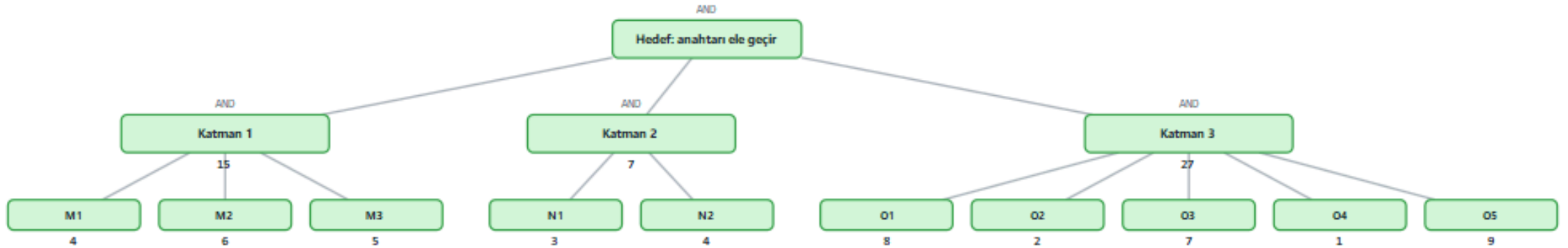
Yaprak	Önlem
Debugger	Anti-debug (6)
Bellek dökümü	Kısa ömür + koruma (6, 10)
Binary'den çıkar	Gizleme + whitebox (9, 11)
Ağdan sız	TLS + pin (10)

Saldırı ağacı: en ucuz yol (animasyon)



Kök bir VEYA düğümü: en ucuz dal Çalışma zamanında bellekten oku (maliyet 1) seçilir — en ucuz yol budur.

Saldırı ağacı — uç durum: her düğüm VE



Kök bir VE düğümü: bütün dallar gerekli -> toplam maliyet = 49. Bu, katmanlı savunmanın gücüdür: her katman saldırganın maliyetine eklenir.

Veri akış diyagramı: beş sembol

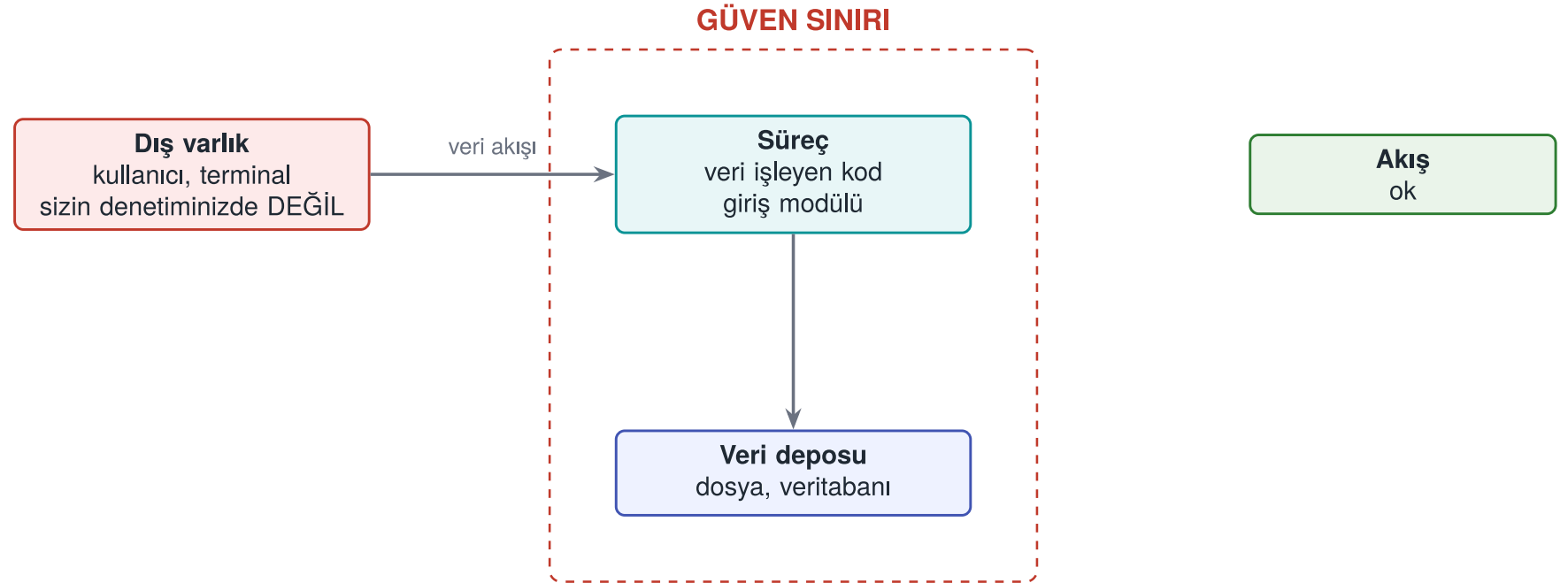
Sembol	Adı	Örnek
□ Dikdörtgen	Dış varlık	Kullanıcı, ödeme terminali
○ Daire	Süreç	Giriş modülü, kriptu kütüphanesi
= İki çizgi	Veri deposu	Veritabanı, yapılandırma dosyası
→ Ok	Veri akışı	HTTP isteği, fonksiyon çağrısı
--- Kesikli çizgi	Güven sınırı	İnternet ↔ sunucu, uygulama ↔ OS

→ Güven sınırını geçen her oku **kırmızıya** boyayın: incelemeye oradan başlayın

Veri akış diyagramı ve güven sınırı — şema

Veri akış diyagramı: dört öge ve güven sınırı

tehdit modellemenin çizim dili



Sınırı geçen her ok bir tehdit adayıdır: orada STRIDE'in altı harfini tek tek sorun.

Kural: güven sınırını çizmeden tehdit listesi çıkarılmaz.

Hangi öğeye hangi harf?

Öge	S	T	R	I	D	E
Dış varlık	✓		✓			
Süreç	✓	✓	✓	✓	✓	✓
Veri deposu		✓	✓ ¹	✓	✓	
Veri akışı		✓		✓	✓	

¹ Denetim kaydıysa · Veri akışının "kimliği" yoktur → **S sorulmaz**

Her harf için soru ve önlem

Harf	Kendinize sorun	Tipik önlem
S	Karşı taraf gerçekten o mu?	Kimlik doğrulama, imza, sertifika sabitleme
T	Değiştirilse fark eder miyim?	MAC/HMAC, imza, bütünlük denetimi
R	"Ben yapmadım" derse kanıtım var mı?	Kurcalamaya dayanıklı günlük, imzalı işlem
I	Diskte, ağda, bellekte, günlükte sızar mı?	Şifreleme, bellek temizleme, maskeleyme
D	Çökertilebilir, tüketilebilir mi?	Girdi sınırı, oran sınırlama, kota
E	Az yetkili, çok yetkilinin işini yaptırabilir mi?	En az ayrıcalık, tam aracılık, bellek güvenliği

Bir taşma = **D** (çöker) + **E** (akış ele geçer) + **I** (bitişik sır okunur)

DFD ögesi -> STRIDE (animasyon)

STRIDE

uygulanan harfler (özet)

Student (browser) -> S,R

Instructor (browser) -> S,R

Login process -> S,T,R,I,D,E

Grade entry process -> S,T,R,I,D,E

Report generator -> S,T,R,I,D,E

Session store -> T,R,I,D

Grade database -> T,R,I,D

Audit log -> T,R,I,D

HTTP login request -> T,I,D

HTTP grade submit -> T,I,D

Grade report response -> T,I,D

Email notification -> T,I,D

12 ögenin hepsi sınıflandırıldı. Her satır, o öge için gerçekten sorulması gereken tehdit sorularını gösteriyor.

STRIDE — uç durum: hepsi veri akışı

STRIDE

uygulanan harfler (özet)

Packet #1 on the wire -> T,I,D

Packet #2 on the wire -> T,I,D

Packet #3 on the wire -> T,I,D

Packet #4 on the wire -> T,I,D

Packet #5 on the wire -> T,I,D

Packet #6 on the wire -> T,I,D

Packet #7 on the wire -> T,I,D

Packet #8 on the wire -> T,I,D

Packet #9 on the wire -> T,I,D

Packet #10 on the wire -> T,I,D

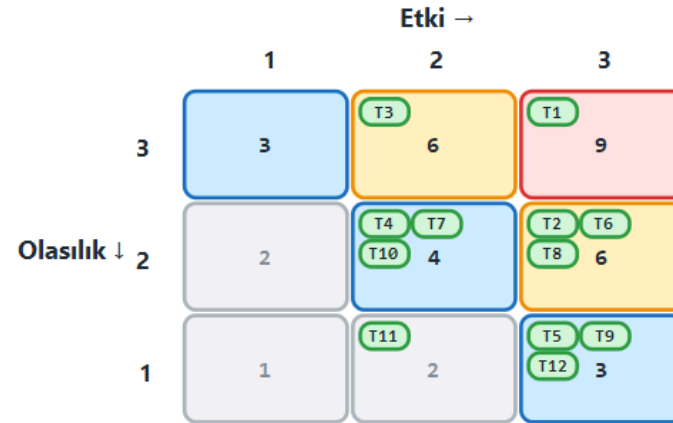
10 ögenin hepsi sınıflandırıldı. Her satır, o öge için gerçekten sorulması gereken tehdit sorularını gösteriyor.

Risk puanı: olasılık $\times$ etki

	Etki 1	Etki 2	Etki 3
Olasılık 3	3	6	9 kritik
Olasılık 2	2	4	6
Olasılık 1	1	2	3

- **Olasılık:** fiziksel erişim mi, internetten mi? Uzmanlık? Otomatikleşir mi?
- **Etki:** hangi varlık, kaç kullanıcı, parasal/hukuki sonuç?
- Daha ayrıntılı: **CVSS** (2. hafta), **saldırı potansiyeli** (13. hafta)

Risk matrisi (animasyon)

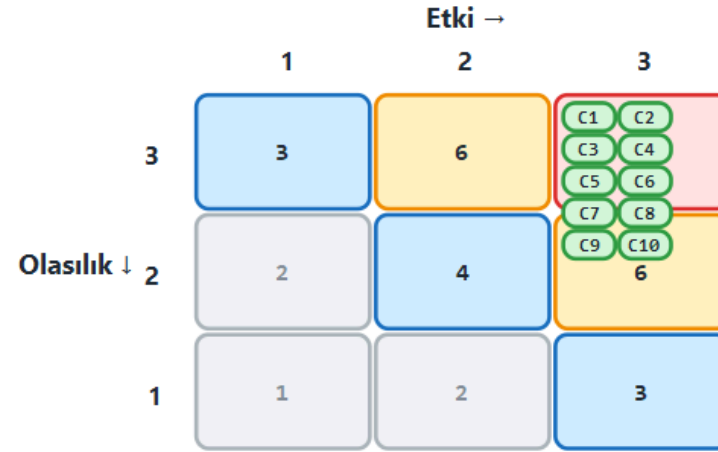


risk sırası (en yüksekte en düşüğe)

1. T1 (9)
2. T2 (6)
3. T3 (6)
4. T6 (6)
5. T8 (6)
6. T4 (4)
7. T7 (4)
8. T10 (4)
9. T5 (3)
10. T9 (3)
11. T12 (3)
12. T11 (2)

12 tehdit puana göre sıralandı. En yüksek puanlı T1, önce ele alınması gereken tehdittir.

Risk matrisi — uç durum: hepsi kritik



risk sırası (en yüksekten en düşüğe)

1. C1 (9)
2. C2 (9)
3. C3 (9)
4. C4 (9)
5. C5 (9)
6. C6 (9)
7. C7 (9)
8. C8 (9)
9. C9 (9)
10. C10 (9)

10 tehdit puana göre sıralandı. En yüksek puanlı C1, önce ele alınması gereken tehdittir.

Tehdide dört yanıt

1. **Azalt** — önlem ekle (en sık)
2. **Ortadan kaldır** — özelliği hiç yazma (en güçlü)
"Parolayı hatırla" yoksa, diskteki parola tehdidi de yok
3. **Aktar** — kart verisini saklama, ödeme sağlayıcısına bırak
4. **Kabul et** — gerekçesiyle **kalan risk** listesine yaz

⚠ Plana yazılmamış risk kabul edilmiş değil, **gözden kaçmıştır**

4. Uygulamalı örnek: "Kasa" parola kasası

Adım 0–1: Uygulama ve kapsam

Kasa: C++ masaüstü parola kasası

- Ana parola → türetilen anahtar → kayıtlar tek dosyada şifreli
- Panoya kopyalama · isteğe bağlı bulut yedek · otomatik güncelleme

Alan	Değer
Değerlendirme hedefi	kasa 1.0 + vault_core + yapılandırma
Kapsam dışı	Yedek sunucusunun kendisi, işletim sistemi
Güvenlik amacı	Ana parolayı bilmeyen için okunamaz ve fark edilmeden değiştirilemez

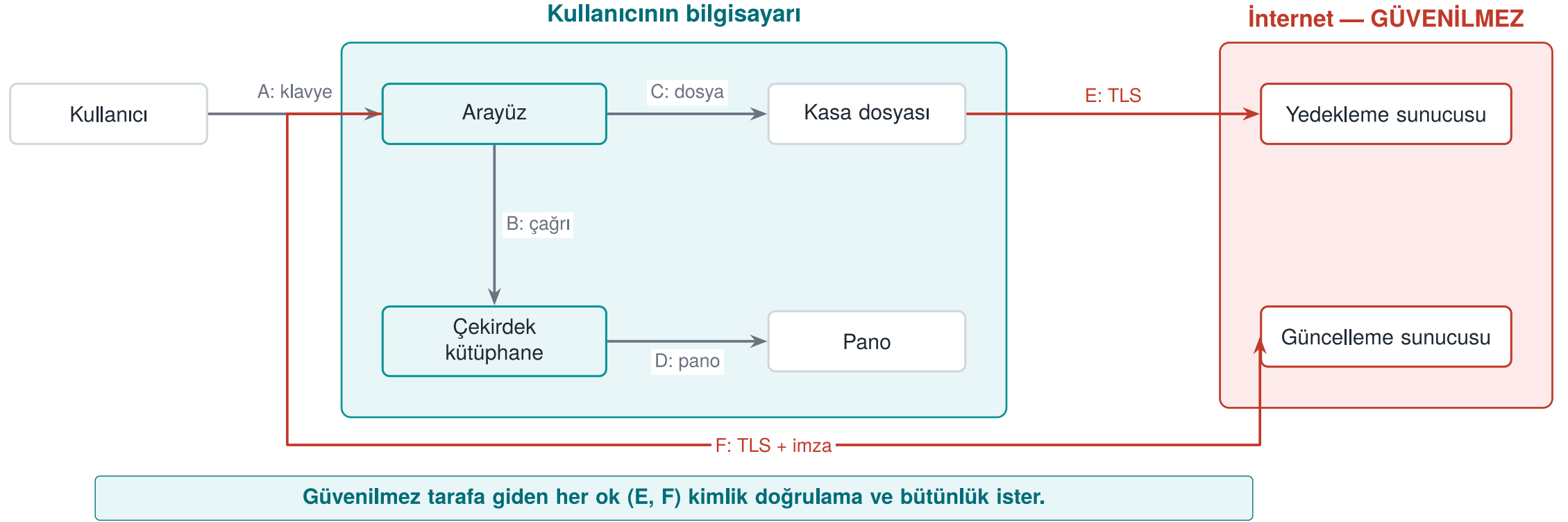
Adım 2: Mimari ve arayüzler

Arayüz	Uçlar	Kimlik doğrulama	Gizlilik / bütünlük
A	Kullanıcı → Arayüz	Ana parola	Ekranda gösterilmez
B	Arayüz → Çekirdek	Aynı süreç	Parola çağrı sonrası silinir
C	Çekirdek → Kasa dosyası	—	AES-GCM, anahtar paroladan
D	Çekirdek → Pano	—	30 s sonra temizlenir
E	Çekirdek → Yedek sunucu	Sertifika + belirteç	TLS + zaten şifreli dosya
F	Arayüz → Güncelleme	Paket imzası	TLS; asıl güvence imza

Kasa mimarisi — şema

Parola kasası: bileşenler ve arayüzler

her ok bir arayüz — her arayüz doğrulanır



Adım 3: Varlıklar

#	Varlık	Nerede	Oluşma → silinme	Koruma
V1	Ana parola	Yalnız bellek	Yazılır → türetilir → hemen silinir	C
V2	Kasa anahtarı	Yalnız bellek	Türetilir → kilitlenince silinir	C, I
V3	Çözülmüş kayıt	Bellek	Görüntülenir → kapanınca silinir	C
V4	Kasa dosyası	Disk, yedek	Kalıcı	C, I
V5	Tuz, türetme parametresi	Dosya başlığı	Oluşturulurken	I
V6	Güncelleme açık anahtarı	Uygulamaya gömülü	Derlemede	I

Koruma sınıfları: **C** gizlilik · **I** bütünlük · **I+** bütünlük + hesap verebilirlik · **N** yok

Adım 4: Tehditler

#	Nerede	Harf	Tehdit	Risk
T1	C / V4	I	Çalınan dosya, zayıf parola çevrimdışı denenir	9
T2	B / V1-2	I	Parola/anahtar bellekte kalır → döküm, takas	6
T3	D / V3	I	Panodaki parolayı başka uygulama okur	6
T5	C / V5	T	Yineleme sayısı 1'e düşürülür	3
T6	F / V6	T, E	Sahte güncelleme kurulur	6
T8	Süreç	T	Kasa ayrıştırıcısında bellek hatası	6

Adım 3-4 (devam): eksik kalan varlık ve tehditler

#	Varlık/Tehdit	Ayrıntı	Değer
V7	Yedekleme belirteci	Disk (OS anahtar deposu); bağlanınca oluşur → iptalde silinir	C
T4	C / V4, T	Kasa dosyası değiştirilir; bozuk kayıt fark edilmeden kullanılır	Risk 4
T7	E, S	Sahte yedekleme sunucusu belirteci çalar	Risk 4
T9	A, S	Ana parola ekrandan görülür (omuz sörfü)	Risk 3

Dokuz tehdidin **hepsi** tabloda olmalı; atlanan tehdit kapatılmamış demektir.

Adım 5: Karşı önlemler

Tehdit	Önlem	Katman
T1	Yavaş türetme (Argon2id / yüksek yinelemeli PBKDF2) + parola gücü şartı	6, 2
T2	Güvenli silme, mlock , döküm kapatma	2
T3	Pano 30 s sonra temizlenir	1
T4-5	Kimlik doğrulamalı şifreleme; başlık = AAD ; en düşük yineleme kodda sabit	6, 2
T6	İmzayı doğrulamadan asla çalıştırma; geri almayı reddet	6
T8	Uzunluk doğrulama + fuzzing + derleyici korumaları	2, 3, 7

Adım 5 (devam): T7 ve T9 önlemleri

Tehdit	Önlem	Katman
T7	Sertifika doğrulama + ana makine adı denetimi, isteğe bağlı sertifika sabitleme	6
T9	Kabul et: ekranda parolayı gizle; fiziksel gözetim kapsam dışı	—

T9 örneği önemli: her tehdide önlem **eklemek** gerekmez, bazen **kabul etmek** doğru karardır.

Adım 6–7: Doğrulama ve kalan risk

Önlem	Test	Beklenen
Türetme	Süre ölçümü	≥ 250 ms
Silme	Kilitten sonra bellek dökümünde parola ara	Bulunmaz
Bütünlük	Dosyanın bir baytını değiştir	"Bozuk" der, açmaz, çökmez
İmza	Bozuk imzalı paket	Reddedilir, günlüğe yazılır

Kalan risk: kasa açıkken aynı kullanıcıyla çalışan zararlı · omuz sörfü · çok zayıf parola

⚠ Testi olmayan önlem değerlendirici gözünde **yoktur**

Adım 6 (devam): T5 ve T8 testleri

Önlem	Test	Beklenen
T5 parametre	Başlıktaki yineleme sayısı 1 yapılır	Uygulama dosyayı reddeder
T8 ayrıştırıcı	24 saat fuzzing (4. hafta)	Çökme yok; sanitazer bulgusu yok

Adım 7 (devam): kalan risk — tam tablo

Risk	Neden kabul edildi?	Yeniden değerlendirme
Kasa açıkken bilgisayarda zararlı çalışıyorsa bellek okunabilir	Aynı kullanıcı yetkisiyle çalışan zararlıya karşı tam koruma mümkün değil; kilitleme süresi kısa tutuldu	Her sürüm
Omuz sörfü (T9)	Fiziksel ortam kapsam dışı	—
Çok zayıf ana parola	Kullanıcı uyarılıyor ama zorla engellenmiyor	Kullanıcı geri bildirimine göre

Sık yapılan hatalar

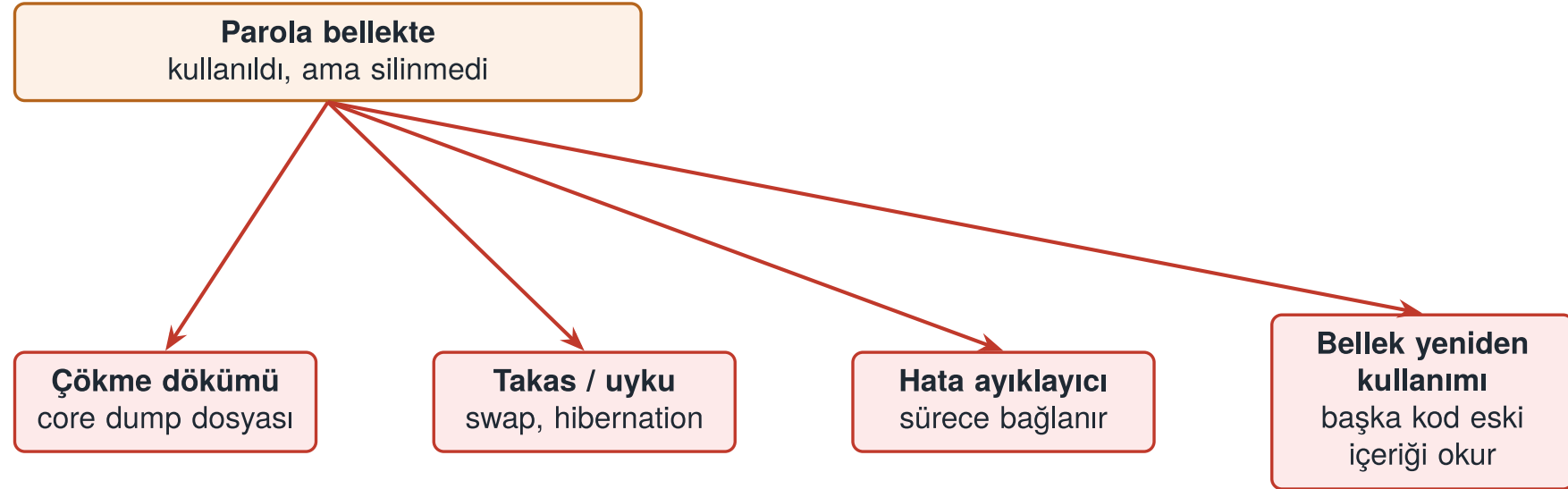
- **Varlık unutmak:** anahtarlar, belirteçler, sayaçlar, yapılandırma, **kodun kendisi**
- **Güven sınırını yanlış çizmek:** kendi istemcinizi "güvenilir" saymak
- **Önlemi tehditle eşleştirmemek:** "TLS kullanıyoruz" — hangi tehdidi kapatıyor, hangisini kapatmıyor?
- **Testsiz önlem:** "siliyoruz" deyip derleyicinin silmeyi kaldırdığını fark etmemek (Demo 2)

5. Program başlarken ve bellekte kalan sırlar

Sır bellekten nasıl sızar — şema

Sır bellekten nasıl dışarı sızar?

değişken kapsam dışına çıkar, baytlar kalır



Çözüm: `explicit_bzero` / `SecureZeroMemory` — derleyicinin kaldıramayacağı silme.

Program boş sayfayla başlamaz

Çalıştıran kişiden **devraldıkları**:

- Ortam değişkenleri: `PATH`, `LD_PRELOAD`, `IFS` ...
- Açık dosya tanıtıcıları, çalışma dizini, `umask`
- Kaynak sınırları, çökme dökümü ayarı

Program bunlara güvenirse, **onları kontrol eden programı kontrol eder.**

Kitap: Viega & Messier, Tarif 1.1–1.9

Demo 1 — PATH ile kandırma (CWE-426)

```
int status = system(COMMAND); /* Linux: "date"  Windows: "hostname" */  
                             /* WHICH ONE runs? */
```

`.\demo.ps1` (Windows) · `sh demo.sh` (WSL / Linux)

STEP 2 - `PATH="$PWD/fake:$PATH" ./bin/linux/report`

Report date:

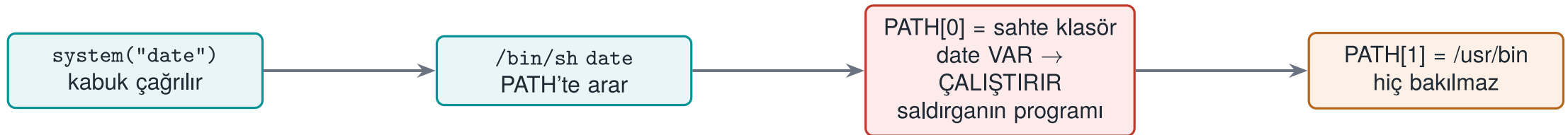
!!! FAKE 'date' ran !!!

STEP 3 - `./bin/linux/report_secure -> Report date: Sat Sep 26 ...`

PATH ile kandırma — şema

PATH ile kandırma: neden system() tehlikeli?

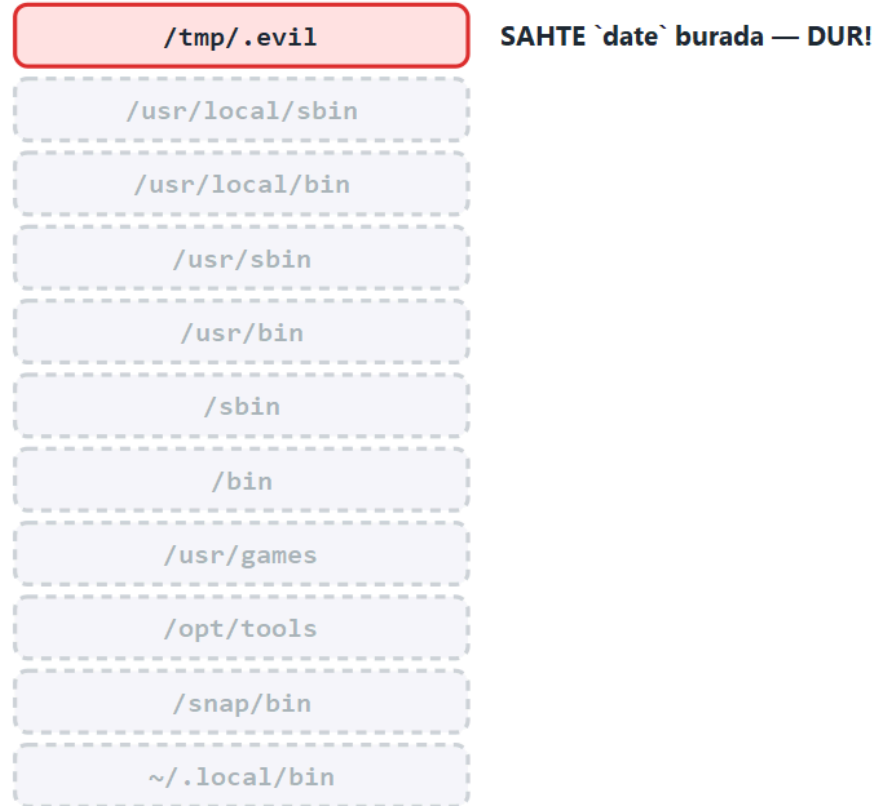
kabuk, programı adına göre PATH sırasıyla arar



Düzeltilme: mutlak yol (/bin/date) · kabuksuz çalıştırma (execve) · temiz ortam.

PATH araması — animasyon (saldırı)

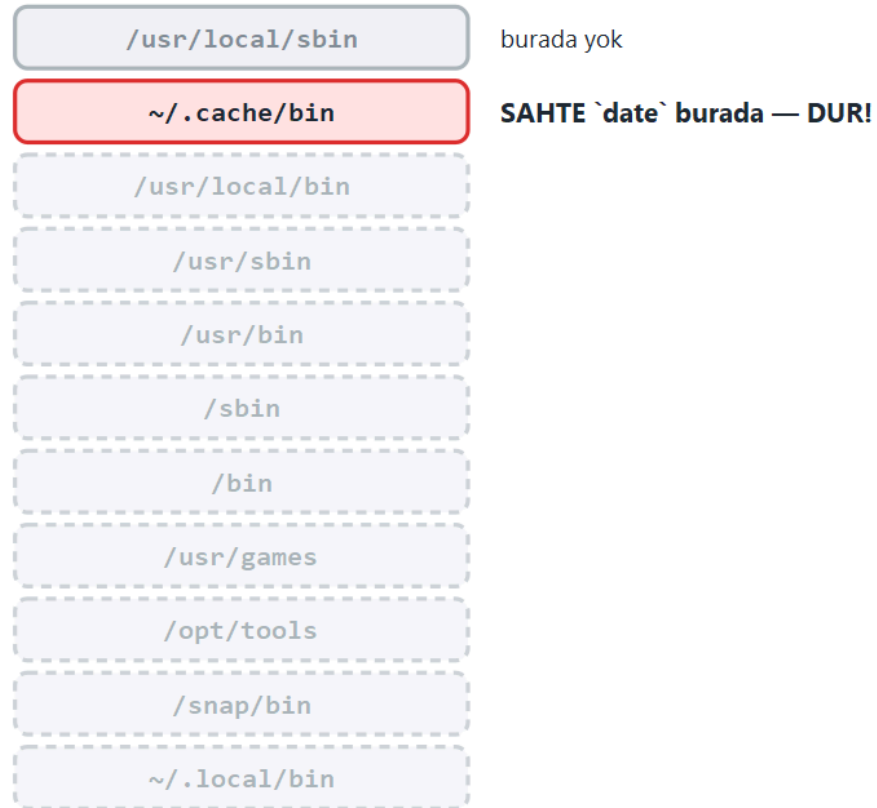
Kabuk `PATH` içinde `date`'i sırayla arıyor:



Kalan 10 dizin hiç kontrol edilmedi — kabuk zaten çalıştıracağı programı bulmuştu.

PATH araması — uç durum: sahte dizin ortada

Kabuk `PATH` içinde `date`'i sırayla arıyor:



Kalan 9 dizin hiç kontrol edilmedi — kabuk zaten çalıştıracağı programı bulmuştu.

Düzeltilme: üç kural

```
char *const args[] = { "date", NULL };  
char *const clean_env[] = { "PATH=/usr/bin:/bin", "LANG=C", NULL };  
posix_spawn(&pid, "/bin/date", NULL, NULL, args, clean_env);
```

1. **Mutlak yol** → PATH araması yok
2. **Kabuk yok** (`posix_spawn` / `execve`) → özel karakter, `IFS` yok
3. **Temiz ortam** → bilinen, küçük bir ortam

Windows: `CreateProcessW` + `System32\hostname.exe` + yalnız `SystemRoot`

En iyisi: başka programa hiç gerek bırakma → `strftime()`, `GetComputerNameW()`

Güvenli başlatma: ne devralınıyor?

Devralınan	Tehlike	Tarif
Ortam değişkenleri	<code>PATH</code> , <code>LD_PRELOAD</code> yüklenecek programı/kütüphaneyi değiştirir	1.1
Açık dosya tanıtıcıları	Ebeveynin gizli dosyası sızar; 0–2 kapalıysa ilk dosya "stdout" olur	1.5
<code>umask</code>	<code>000</code> → herkese yazılabilir dosyalar	2.7
Kaynak sınırları	Çekirdek dökümü açık → bellek diske	1.9
Kimlik ve yetkiler	setuid: gereğinden fazla yetki	1.3

Kural: kendi kurmadığın hiçbir şeye güvenme

Programın devraldığı durum — şema

Program başlarken ebeveyninden ne devralır?

`main()`'in ilk satırlarında yapılacak iş budur

Ortam değişkenleri

PATH, LD_PRELOAD, LD_LIBRARY_PATH — hangi kod yüklenecek

Açık dosya tanıtıcıları

0/1/2 kapalıysa açtığınız dosya stdin olur

umask ve çalışma klasörü

oluşturduğunuz dosyanın izinleri buradan gelir

Sinyal düzeni ve kaynak sınırları

yok sayılan sinyal, düşük core limiti

Gerçek ve etkin kullanıcı kimliği

setuid ise ayrıcalık elinizdedir

Hiçbirine güvenmeyin: gerekeni kendiniz kurun, gerekmeyeni kapatın.

Devralınan diğer riskler: çalışma dizini, sinyal, dlopen

- **Çalışma dizini:** Göreli yollar (`./ayar.ini`) saldırganın seçtiği bir klasöre işaret edebilir
- **Sinyal ayarları:** Ebeveynin yok saydığı sinyaller **çocukta da** yok sayılır
- **`dlopen("lib.so")` :** `LD_LIBRARY_PATH` ve arama yollarından etkilenir → tam yol kullanın; `LD_*` Adım 1'de temizlenmiş olmalı

Kural aynı: **programın kendisinin kurmadığı hiçbir şeye güvenme.**

Adım 1–2: Ortam ve tanıtıcılar

```
/* Environment: an ALLOWLIST, not a blocklist */
clearenv();                               /* copy what you want to keep first! */
setenv("PATH", "/usr/bin:/bin", 1);

/* if 0-2 are closed, bind them to /dev/null, close the rest */
for (int fd = 0; fd <= 2; fd++)
    if (fcntl(fd, F_GETFD) == -1 && open("/dev/null", O_RDWR) != fd) abort();
close_range(3, ~0U, 0);                   /* Linux 5.9+ */
```

- `getenv` işaretçisi `clearenv` sonrası **geçersiz**
- Kendi dosyalarınızda her zaman `O_CLOEXEC`

Adım 3–4: İzinler ve çökme dökümü

```
umask(077);                                /* rw----- */
int fd = open(path, O_WRONLY | O_CREAT | O_EXCL, 0600);

struct rlimit r = { 0, 0 };
setrlimit(RLIMIT_CORE, &r);                /* dump size 0 */
prctl(PR_SET_DUMPABLE, 0, 0, 0, 0);        /* + cannot be attached to via ptrace */
```

- Windows: `SetErrorMode(SEM_FAILCRITICALERRORS | SEM_NOGPFAULTERRORBOX)`
- Asıl savunma: döküm alınsa bile içinde **sır kalmamalı**

Adım 5: Yetkiyi kalıcı bırak ve doğrula

```
if (setresgid(g, g, g) != 0) abort(); /* 1. GROUP first */
if (setresuid(u, u, u) != 0) abort(); /* 2. then the user: all three r/e/s */
getresuid(&r, &e, &s);
if (r != u || e != u || s != u) abort(); /* 3. VERIFY */
```

- `seteuid()` geçicidir: saklı kimlik yetkili kalır (2. hafta Demo 13)
- Windows: **kısıtlanmış belirteç** (`CreateRestrictedToken`)
- Daha iyisi: **ayrıcalık ayrımı** — küçük yetkili süreç + büyük yetkisiz süreç (Tarif 1.4)

Adım 6: Program ve kütüphane yüklerken

❌ Hatalı	✅ Doğrusu
<code>system("convert " + file)</code>	<code>execve("/usr/bin/convert", argv, clean_env)</code>
<code>execvp("convert", ...)</code>	Tam yolla <code>execve</code>
<code>CreateProcess(NULL, "C:\Program Files\...")</code>	<code>lpApplicationName</code> = tam yol, komut satırı tırnaklı
<code>LoadLibrary("helper.dll")</code>	<code>SetDefaultDllDirectories(LOAD_LIBRARY_SEARCH_DEFAULT_DIRS)</code> + tam yol

Windows DLL arama sırası = `PATH` sorununun ikizi (CWE-427)

Hepsi bir arada: `secure_startup()`

```
int main(int argc, char **argv)
{
    sanitize_file_descriptors(); /* 1. descriptors */
    sanitize_environment();      /* 2. environment */
    umask(077);                  /* 3. permissions */
    disable_core_dump();         /* 4. dump */
    drop_privileges();           /* 5. privilege */
    /* ... the actual work ... */
}
```

Sıra önemli: başka hiçbir şey yapmadan önce

Güvenli başlatma boru hattı (animasyon)

devralınan ortam (12 değişken)

<code>PATH=/tmp/evil:/usr/bin:/bin</code>
<code>LD_PRELOAD=/tmp/keylogger.so</code>
<code>IFS=\$'\n'</code>
<code>HOME=/home/alice</code>
<code>TZ=Europe/Istanbul</code>
<code>LANG=en_US.UTF-8</code>
<code>EDITOR=vim</code>
<code>SHELL=/bin/bash</code>
<code>TERM=xterm-256color</code>
<code>PWD=/home/alice/vault</code>
<code>USER=alice</code>
<code>BASH_ENV=/tmp/payload.sh</code>

yeni, bilinen ortam

<code>PATH=/usr/bin:/bin</code>
<code>HOME=/home/alice</code>
<code>TZ=Europe/Istanbul</code>
<code>LANG=en_US.UTF-8</code>
<code>umask: 000 -> 077</code>
<code>çökme dökümü: açık -> kapalı</code>
<code>yetki: root -> user</code>

Sonuç: 12 devralınan değişkenden yalnız 4 tanesi hayatta kaldı (+ zorla ayarlanan PATH); tanıtıcılar, umask, döküm ve yetki hepsi bilinen güvenli değerlerde.

Güvenli başlatma — uç durum: hepsi tehlikeli

devralınan ortam (11 değişken)

PATH=danger-0
LD_PRELOAD=danger-1
LD_LIBRARY_PATH=danger-2
IFS=danger-3
BASH_ENV=danger-4
ENV=danger-5
PATH=/tmp/evil0
PATH=/tmp/evil1
PATH=/tmp/evil2
PATH=/tmp/evil3
PATH=/tmp/evil4

yeni, bilinen ortam

PATH=/usr/bin:/bin
umask: 000 -> 077
çökme dökümü: açık -> kapalı
yetki: root -> user

Sonuç: 11 devralınan değişkenden yalnız 1 tanesi hayatta kaldı (+ zorla ayarlanan PATH); tanıtıcılar, umask, döküm ve yetki hepsi bilinen güvenli değerlerde.

Demo 2 — Bellekte kalan parola (CWE-14)

```
char password[64];
n = file_read(fd, password, sizeof(password) - 1);
key = derive_key(password, n);
memset(password, 0, sizeof(password));          /* version 2 */
explicit_bzero(password, sizeof(password));      /* version 3, Linux */
SecureZeroMemory(password, sizeof(password));   /* version 3, Windows */
```

```
Never wipes      -> password FOUND in the dump
memset (-02)     -> password FOUND in the dump      ← !!!
explicit_bzero   -> password not found in the dump
```

Neden memset kayb oldu?

Derleyicinin düşüncesi:

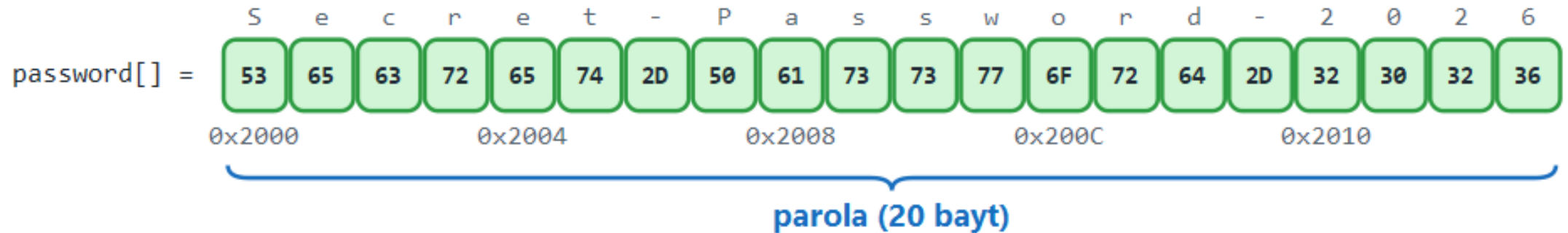
" password bundan sonra **hiç okunmuyor**. Ona sıfır yazmak sonucu değiştirmez. **Gereksiz — siliyorum.**"

Kullanılmayan yazmanın silinmesi (dead store elimination): hız için iyi, **sırlar için felaket**.

Platform	Kaldırılmayan silme
Linux (glibc)	<code>explicit_bzero</code>
Windows	<code>SecureZeroMemory</code>
C11 Ek K	<code>memset_s</code>
OpenSSL	<code>OPENSSL_cleanse</code>

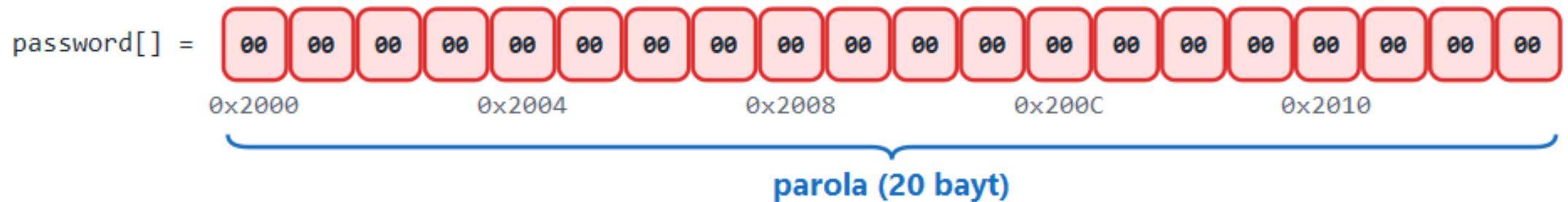
+ `setrlimit(RLIMIT_CORE, 0)` · `mlock` · sırrı **kısa** tut, **kopyalama**

Parola bellekte (animasyon)



Şimdi bellek dökümü alınsa: parola HÂLÂ orada — "RESULT: password FOUND in the dump".

Parola bellekte — uç durum: gerçek silme



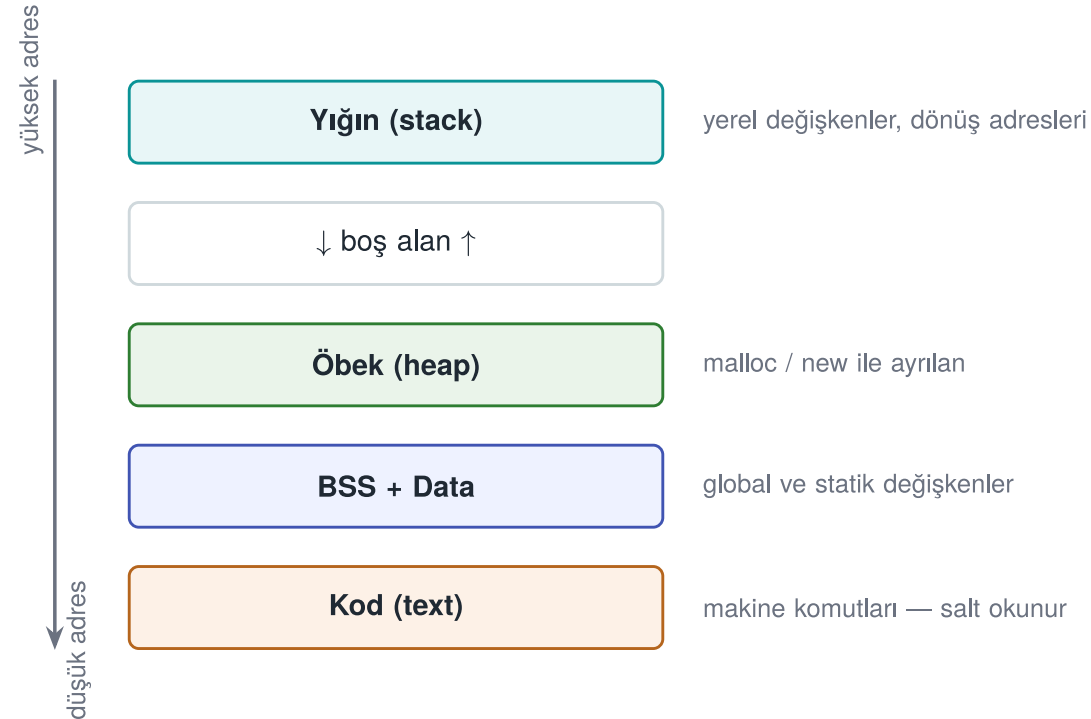
Şimdi bellek dökümü alınsa: parola artık orada değil — "RESULT: password not found in the dump".

6. Süreç belleği ve taşmalar

Süreç belleği

Süreç belleği: verileriniz nerede duruyor?

yığın aşağı, öbek yukarı büyür — ortada boşluk



Taşmanın sonucu hangi bölgede olduğuna bağlıdır: yığında dönüş adresi, öbekte komşu blok.

C dizinin sonunu denetlemez. `name[16]` 'ya 17. bayt → arkadaki değişkenin üzerine.

Yapının bellekteki hali

```
struct session { char name[16]; int admin; };
```

```
offset: 0 ..... 15 | 16  17  18  19
        [ a l i c e \0 . . . . ] [ 00 00 00 00 ]  admin = 0

"AAAAAAAAAAAAAAAAAB" (17 characters) copied in:
        [ A A A A A A A A A A A A ] [ 42 00 00 00 ]  admin = 66 (!)
                                'B'  '\0'
```

Az anlamlı bayt önce (little-endian): en düşük bayt en düşük adreste.

Süreç belleği (animasyon)



Sonuç: text=2, data=3, bss=1, heap=1, stack=3. Aynı fonksiyonun içinde bile değişkenler farklı bölgelerde yaşayabilir — bölgeyi belirleyen TÜR değil, BİLDİRİM biçimidir.

Süreç belleği — uç durum: hepsi öbekte



Sonuç: text=0, data=0, bss=0, heap=10, stack=0. Aynı fonksiyonun içinde bile değişkenler farklı bölgelerde yaşayabilir — bölgeyi belirleyen TÜR değil, BİLDİRİM biçimidir.

Bellekteki yapı ve taşma — şema

Yapının bellekteki hâli: taşma neyi ezer?

ad[16] ve yönetici yan yana durur

ÖNCE

"ayse"
ad[16]

00 00 00 00
yönetici = 0

SONRA

"AAAAAAAAAAAAAAAAAAAA"
17 karakterlik "AAAA...AAAB" kopyalandı

B 00 00 00
yönetici $\neq$ 0

Tek bayt taşma yetkiyi değiştirdi: sınır denetimi olmayan kopya, komşu alanı ezer.

Arabellek taşması nedir?

Bir diziye **ayrılan boyuttan fazla** veri yazmak → taşan baytlar **komşu değişkenlerin** üzerine
C neden bu kadar kolay taşar?

1. Dizi **boyutunu bilmez**
2. Fonksiyona yalnız **başlangıç adresi** gider
3. Dize (string) = `'\0'` ile biten dizi; `strcpy` hedefi **sormaz**
4. Derleyici sınır denetimi **eklemez**

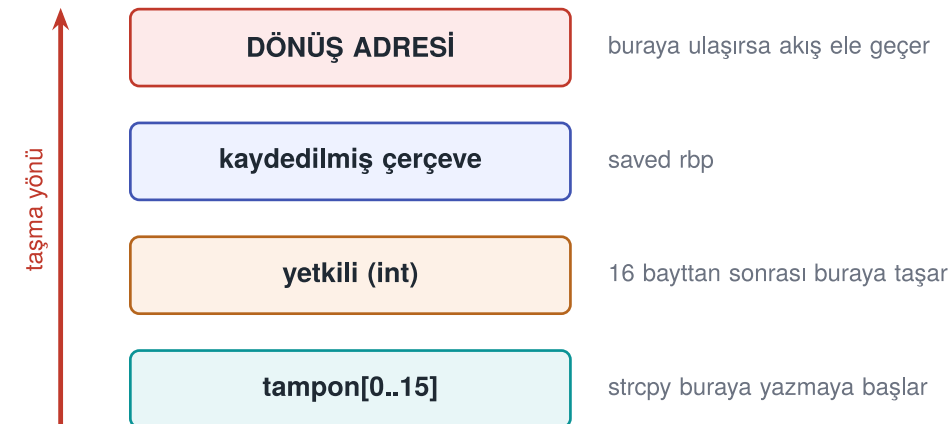
CWE-787 "sınırlar dışına yazma": CWE Top 25'in hep ilk sıralarında

Yığında bir taşma

```
void greet_user(const char *name) {  
    int authorized = 0;  
    char buffer[16];  
    strcpy(buffer, name);    /* name > 15 characters? */  
}
```

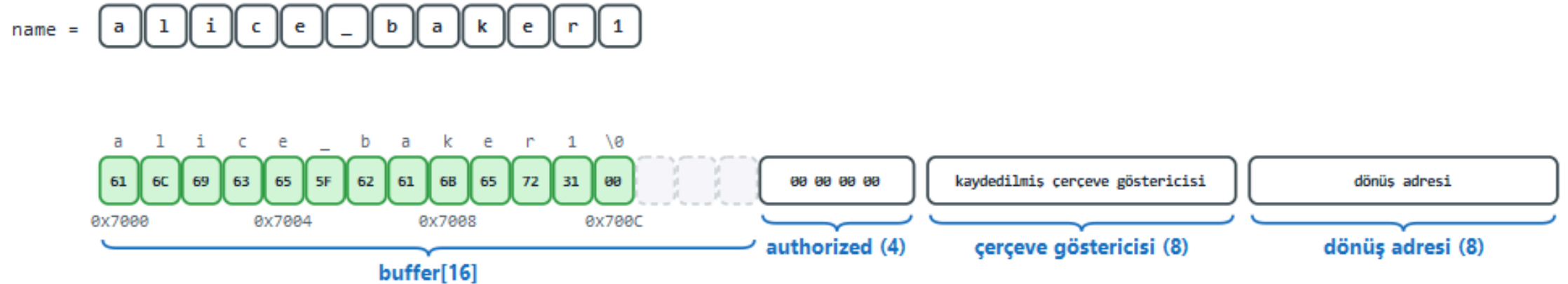
Yığında taşma adım adım

strcpy aşağıdan yukarı yazar



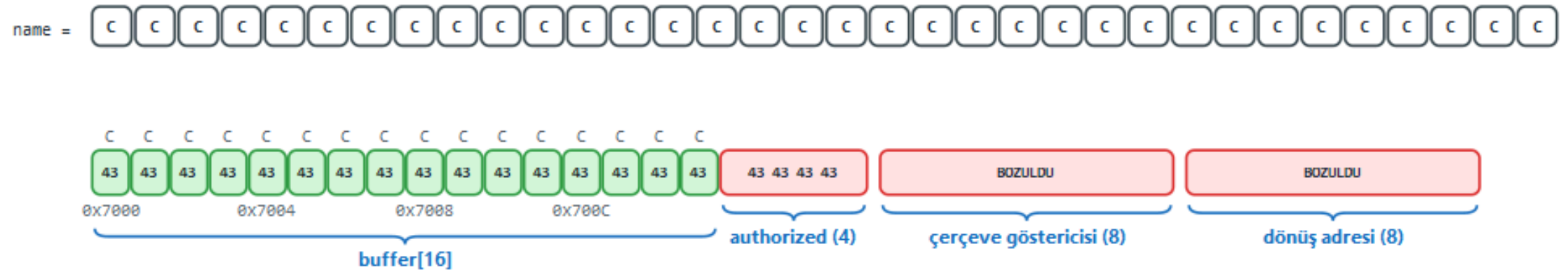
Kanarya, dönüş adresinden önce durur: ezilirse program sonlanır (ama yetkili zaten ezilmiştir).

Yığın çerçevesi (animasyon)



12 karakter 16 baytlık buffer'a rahatça sığıyor; taşma yok, authorized = 0, program doğru çalışır.

Yığın çerçevesi — uç durum: dönüş adresi eziliyor



Fonksiyon return yapmaya çalışır: dönüş adresi geçerli bir kod adresi değil -> çökme (korumasız bir sistemde akış ele geçirilebilir).

Taşma türleri

Tür	Tipik neden	CWE
Yığın (stack) taşması	<code>strcpy</code> , <code>gets</code> , sınırsız <code>scanf("%s")</code>	121
Öbek (heap) taşması	Yanlış uzunluklu <code>memcpy</code>	122
Bir fazla (off-by-one)	<code><=</code> yerine <code><</code> ; <code>'\0'</code> için yer yok	193
Sınır dışı okuma	Karşı tarafın bildirdiği uzunluğa güvenmek	125
Tamsayı kaynaklı	<code>n * boyut</code> taşar → küçük blok, büyük kopya	190 → 787

Heartbleed (2014): hiçbir şey yazılmadı

- İstemci: "1 bayt gönderiyorum, **64 KB** diye bildiriyorum"
 - Sunucu: bildirilen uzunluğu **denetlemeden** 64 KB geri yolladı
 - Sızan: parolalar, oturumlar, hatta sunucunun **özel anahtarı**
 - CVE-2014-0160 · CWE-125 · Düzeltme: **tek bir uzunluk denetimi**
- ➔ Sınır dışı okuma da yazma kadar tehlikeli; üstelik **çökmez, iz bırakmaz**

Bir fazla hatası

```
char name[8];
for (int i = 0; i <= 8; i++)      /* 9 times: name[8] is out of bounds */
    name[i] = source[i];

strncpy(name, source, sizeof name); /* source >= 8 -> no '\0' */
printf("%s\n", name);              /* keeps reading past the end */
```

- `strncpy` güvenli `strcpy` **değildir**
- Tek bayt taşma bile komşu değişkenin en düşük baytını değiştirir

Tehlikeli fonksiyonlar → yerine

✗	✓
<code>gets(s)</code>	<code>fgets(s, sizeof s, stdin)</code> — <code>gets</code> C11'de kaldırıldı
<code>strcpy(d, s)</code>	Uzunluk denetimi + <code>memcpy</code> , <code>snprintf</code> , <code>strncpy</code> (glibc 2.38+)
<code>strcat(d, s)</code>	Kalan yeri hesapla; <code>snprintf</code> , <code>strlcat</code>
<code>sprintf(d, ...)</code>	<code>snprintf(d, sizeof d, ...)</code> + dönüş değeri denetimi
<code>scanf("%s", s)</code>	<code>scanf("%15s", s)</code>
<code>memcpy(d, s, n)</code>	Önce <code>n <= sizeof d</code>
<code>char a[n]</code> (VLA), <code>alloca</code>	Sabit üst sınır ya da <code>malloc</code>

Tarif 3.3–3.4 bugün: `snprintf` / `strncpy`, MSVC `strcpy_s`, C++ `std::string` / `span`

Doğru kalıp: önce doğrula, sonra sınırlı kopyala

```
int greet_user(const char *name)
{
    char buffer[16];
    size_t n = strlen(name, sizeof buffer); /* read at most 16 bytes */
    if (n == sizeof buffer)                 /* too long: REJECT */
        return -1;
    memcpy(buffer, name, n);
    buffer[n] = '\0';                       /* always terminate */
    printf("Hello %s\n", buffer);
    return 0;
}
```

Neden kesmek yerine reddetmek? `/home/ayse/gizli_rapor.txt` → `/home/ayse/gizli`

C++: boyutu tür taşır — ama dikkat

```
void greet_user(const std::string &name) {  
    if (name.size() > 15) throw std::invalid_argument("name too long");  
    std::cout << "Hello " << name << '\n';  
}
```

- `vector::operator[]` sınır **denetlemez** → `at()` denetler
- `c_str()` ile alınan işaretçiye yazmak → C'nin bütün sorunları geri gelir

Taşmayı kim yakalar?

Katman	Araç	Ne zaman?
Kod	Sınır denetimi, güvenli API	Her zaman
Uyarı	<code>-Wall -Wextra -Wformat-security · /W4 /sdl</code>	Geliştirme
Statik analiz	<code>clang --analyze , cppcheck , /analyze</code>	CI
Sanitizer	<code>-fsanitize=address · /fsanitize=address</code>	Test
Fuzzing	libFuzzer, AFL++	Test (4. hafta)
Derleyici	Kanarya <code>-fstack-protector-strong · /GS , _FORTIFY_SOURCE</code>	Sürüm
OS / donanım	DEP/NX, ASLR · Intel CET, ARM PAC	Sürüm

⚠️ Koruma sömürüyü **zorlaştırır**, hatayı **gidermez** — kanarya programı sonlandırır = hizmet engelleme

ASan raporunu okumak

```
ERROR: AddressSanitizer: stack-buffer-overflow
WRITE of size 21 at 0x7ffd... thread T0
    #0 in strcpy
    #1 in greet_user example.c:7
    #2 in main example.c:15
This frame has 1 object(s):
    [32, 48) 'buffer' <== Memory access at offset 48 overflows this variable
```

1. **Ne?** yığın taşması, 21 bayt yazma
2. **Nerede?** `example.c:7`, `strcpy`
3. **Hangi değişken?** `buffer` (16 bayt), erişim tam sınırın ötesinde

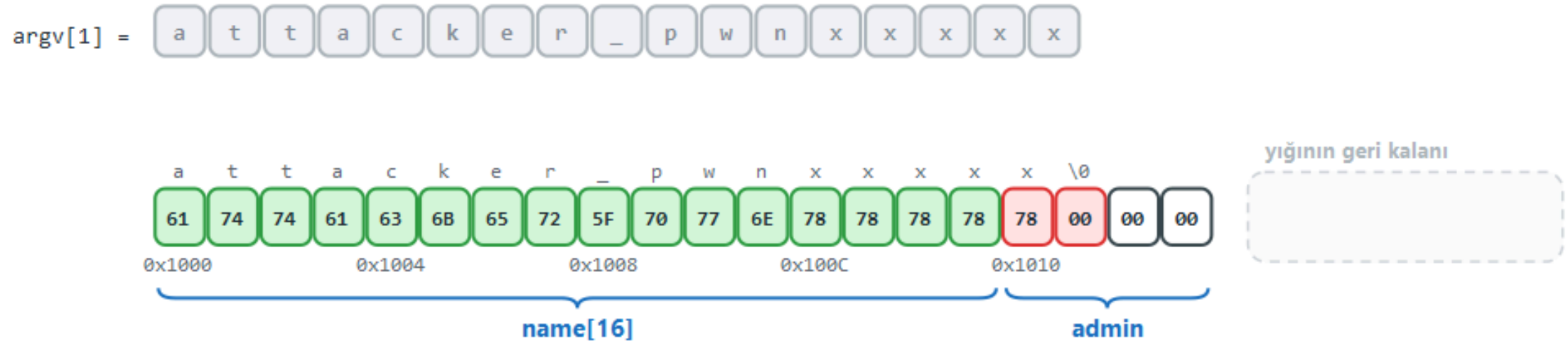
Yaklaşık 2× yavaş → yalnız **test** derlemesinde

Demo 3 — Taşmayla yetki yükseltme (CWE-121)

`.\demo.ps1` (Windows) · `sh demo.sh` (WSL / Linux)

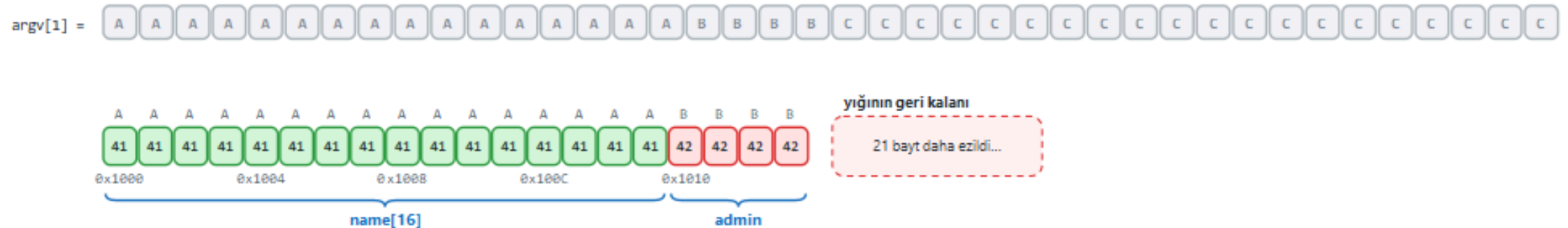
Sürüm	17 karakterlik saldırı
<code>login</code> (korumasız)	✗ YÖNETİCİ erişimi verildi
<code>login_asan</code> (ASan)	✗ sessiz kaldı — taşma yapının içinde
<code>login_asan</code> + 40 karakter	✓ stack-buffer-overflow yakalandı
<code>login_checked</code> (<code>_FORTIFY_SOURCE=2</code> / <code>strcpy_s</code>)	✓ program durduruldu
<code>login_secure</code>	✓ Reddedildi

Taşma — animasyon (17 bayt, **admin** ayarlanıyor)



Sonuç: admin = 120 (0x00000078) — sıfır değil, erişim VERİLDİ.

Taşma — uç durum: struct'ın ötesine geçiyor (40 bayt, ASan)



Sonuç: admin = 1111638594 (0x42424242) — sıfır değil, erişim VERİLDİ. Üstelik yığında struct session'ın ötesi de ezildi (ASan bunu "stack-buffer-overflow" olarak bildirir).

Düzeltilme: doğrula, sonra sınırlı kopyala

```
/* Allow-list: 1-15 characters, letter/digit/_/- */
static int name_is_valid(const char *s)
{
    size_t n = strlen(s, 16);
    if (n == 0 || n >= 16) return 0;
    for (size_t i = 0; i < n; i++) {
        unsigned char c = (unsigned char)s[i];
        if (!isalnum(c) && c != '_' && c != '-') return 0;
    }
    return 1;
}

struct session s = { .admin = 0 }; /* secure default */
snprintf(s.name, sizeof(s.name), "%s", argv[1]); /* a copy that knows the size */
```

Canary, NX, ASLR (4. hafta) **sömürmeyi zorlaştırır, hatayı düzeltmez.**

Tamsayılar: -1 ne zaman dev olur?

`int` işaretli · `size_t` işaretsiz · negatifler **ikiye tümleyen**

```
int    -1    = 0xFFFFFFFFFFFFFFFF
size_t      = 18 446 744 073 709 551 615
```

```
if (length > 16) return -1; /* -1 PASSES this check */
memcpy(dest, source, length); /* int -> size_t */
```

Demo 4 — İşaretli uzunluk (CWE-195)

```
STEP 0  GCC -Wall -Wextra: silent · -Wsign-conversion: warning  
        MSVC (C): silent · same code as C++: C4365  
STEP 1  copy 8      -> Copied: SAMPLE-R  
STEP 2  copy 100    -> Rejected  
STEP 3  copy -1     -> Linux: SIGSEGV (139) · Windows: 0xC0000409  
STEP 4  ASan        -> negative-size-param: (size=-1)  
STEP 5  copy_secure -1 / 12abc -> Rejected
```

Kural: boyutlar `size_t` · alt ve üst sınır · `strtol` · uyarıları aç

İşaretli uzunluk hatası — şema

İşaretli uzunluk hatası: aynı bitler, iki farklı sayı

int işaretli, size_t işaretsiz — dönüşüm sessizdir

int uzunluk = -1
programcının niyeti:
“geçersiz” işareti

aynı bit dizisi

0xFFFFFFFFFFFFFFFF
64 bit, ikiye tümleyen

`memcpy(..., uzunluk)` — size_t bekler

(size_t) 18.446.744.073.709.551.615
sınır denetimi “uzunluk < 16” ile
yapılmışsa, o denetim -1’i GEÇİRİR

Kural: boyutlar baştan sona size_t tutulur; -Wsign-conversion açılır.

İşaretli uzunluk taraması (animasyon)

copy_record(dest, source, length) — 10 çağrı
uzunluk

5	size_t = 5
16	size_t = 16
17	reddedildi (uzunluk > 16)
-1	size_t = 18446744073709551615
8	size_t = 8
0	size_t = 0
12	size_t = 12
20	reddedildi (uzunluk > 16)
-5	size_t = 18446744073709551611
16	size_t = 16

10 çağrının 2 tanesi negatif uzunlukla geçti ve devasa bir size_t'ye dönüştü — tek bir alt sınır denetimi (length < 0) hepsini önlerdi.

İşaretsiz uzunluk — uç durum: hepsi negatif

copy_record(dest, source, length) — 10 çağrı
uzunluk

-1	size_t = 18446744073709551615
-2	size_t = 18446744073709551614
-3	size_t = 18446744073709551613
-4	size_t = 18446744073709551612
-5	size_t = 18446744073709551611
-8	size_t = 18446744073709551608
-16	size_t = 18446744073709551600
-32	size_t = 18446744073709551584
-64	size_t = 18446744073709551552
-128	size_t = 18446744073709551488

10 çağrının 10 tanesi negatif uzunlukla geçti ve devasa bir size_t'ye dönüştü — tek bir alt sınır denetimi (length < 0) hepsini önledi.

7. Bellek yönetimi, bölümlleme ve güvenli süreç

Dinamik belleğin yaşam döngüsü

Ayr → doğrula → kullan → sırrı sil → bir kez serbest bırak → işaretçiyi unut

Adım	Unutulursa	CWE
Doğrula	<code>malloc</code> NULL döner, denetlenmez → çökme	476
Kullan	Sınır dışı yazma	787
Sırrı sil	Blok bir sonraki <code>malloc</code> 'la başka koda gider	226
Bir kez bırak	Çift serbest bırakma	415
İşaretçiyi unut	Serbest bırakıldıktan sonra kullanma (UAF)	416
Hiç bırakmamak	Sızıntı → hizmet engelleme	401

Belleğin dört ömrü

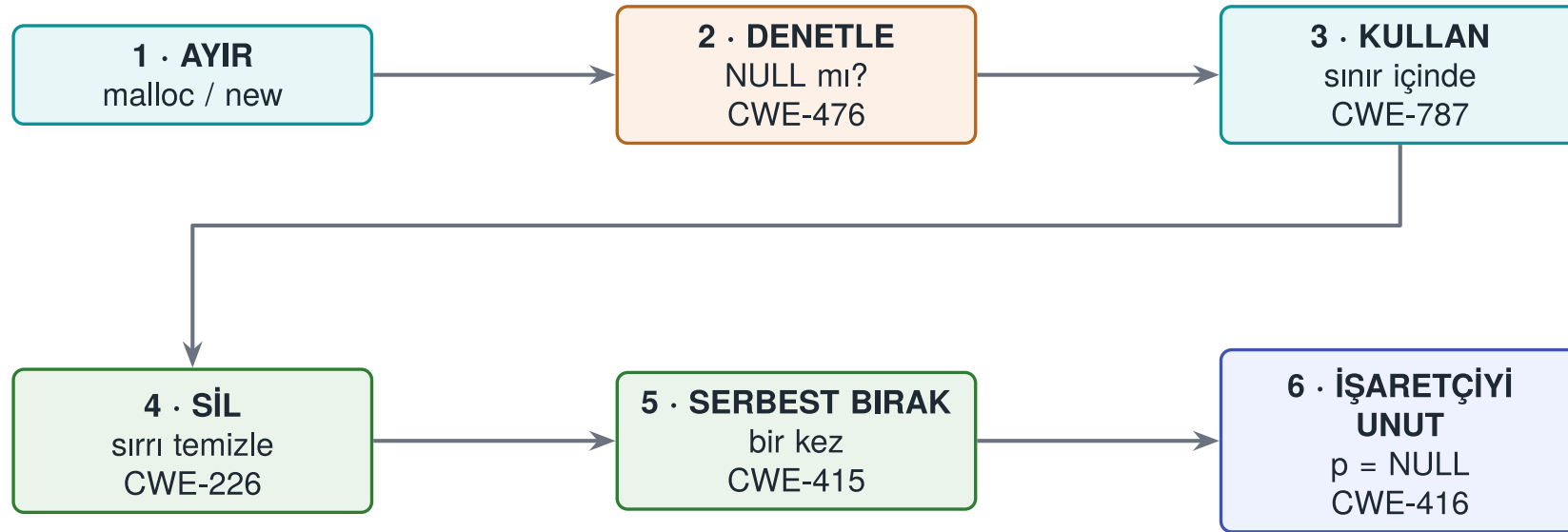
Ömür	Nasıl oluşur?	Ne zaman biter?	Tipik hata
Statik	Genel değişken, <code>static</code> yerel	Program bitince	Aynı anda birden fazla iş parçacığının yazması
Otomatik	Fonksiyon içi yerel değişken	Fonksiyon dönünce	Yerel değişkenin adresini döndürmek
Dinamik	<code>malloc</code> / <code>calloc</code> / <code>new</code>	<code>free</code> / <code>delete</code> çağrılınca	Sızıntı, çift serbest bırakma, UAF
İş parçacığı	<code>_Thread_local</code> / <code>thread_local</code>	İş parçacığı bitince	—

Bu haftanın taşma ve UAF örnekleri hep **dinamik** ömürle ilgili; sırf onu bilmek yetmez.

Belleğin ömrü ve CWE'ler — şema

Belleğin dört ömrü ve her adımın CWE'si

her adımın atlanması ayrı bir zafiyet sınıfı üretir



Altı adımın hepsi yapılmazsa bellek hatası kaçınılmazdır.

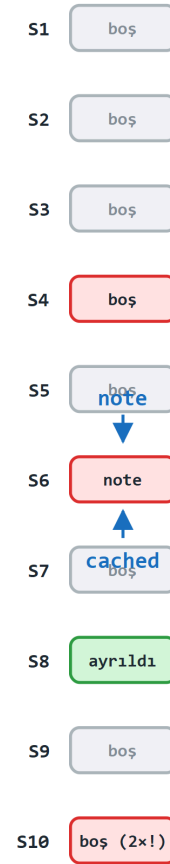
UAF: neden bu kadar tehlikeli?

```
User *active = malloc(sizeof *active);
User *cached = active;                /* a second owner */
free(active); active = NULL;          /* cached is still at the old address */

char *note = malloc(sizeof(User));    /* the allocator may hand back the same block */
strcpy(note, "....");
if (cached->privilege) { ... }         /* reading 'note's bytes */
```

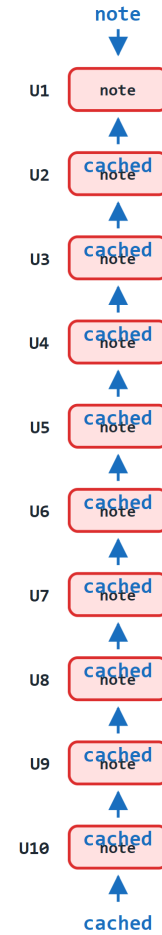
- `free` belleği OS'e vermez, **boş listeye** koyar → aynı blok yeniden verilir
-  **Sahiplik kuralı:** her bloğun **tek sahibi** var, yalnız o serbest bırakır

Bellek hataları turu (animasyon)



10 oturum: 5 temiz, 2 szıntı, 2 çift-serbest-bırakma, 1 UAF.

Bellek hataları — uç durum: hepsi UAF



10 oturum: 0 temiz, 0 sızıntı, 0 çift-serbest-bırakma, 10 UAF.

Düzeltilmiş kalıplar

```
Record *k = calloc(n, sizeof(Record));          /* checks the multiplication for overflow + zeroes */
if (k == NULL) return ERR_MEMORY;

Record *resized = reallocarray(k, n2, sizeof(Record));
if (resized == NULL) { free(k); return ERR_MEMORY; } /* do not lose k */
k = resized;
```

✗ `buffer = realloc(buffer, n);` → başarısızsa eski blok kaybolur

✗ Sır tutan tamponda `realloc` → taşınan eski blok **sıfırlanmaz**

Sırları silmek (Tarif 13.2)

Platform	Fonksiyon
Linux, BSD	<code>explicit_bzero(p, n)</code>
Windows	<code>SecureZeroMemory(p, n)</code>
C11 Ek K / C23	<code>memset_s</code> / <code>memset_explicit</code>
Hiçbiri	<code>volatile</code> işaretçiyle bayt bayt

Sırrın **gizli kopyaları**: değerle geçirilen yapılar · `realloc` 'un eski bloğu · `std::string` (SSO, silmeden yok olur) · G/Ç tamponları · takas ve hazırda bekletme dosyası

Diske düşmesin (Tarif 13.3) ve RAI

```
mlock(key, sizeof key);          /* Windows: VirtualLock */  
/* ... use it ... */  
secure_wipe(key, sizeof key);  
munlock(key, sizeof key);
```

```
class SecretKey {                  /* destructor wipes automatically */  
    ~SecretKey() { secure_wipe(v_.data(), v_.size()); }  
    SecretKey(const SecretKey&) = delete; /* copy = a second secret */  
};
```

Kilitleme **sayfa** düzeyinde, miktar sınırlı → yalnız küçük anahtar bölgesi

Bellek hatası avcıları

Araç	Bulduğu	Kullanım
AddressSanitizer	Taşma, UAF, çift bırakma	<code>-fsanitize=address</code> · <code>/fsanitize=address</code>
LeakSanitizer	Sızıntı	ASan ile (Linux)
MemorySanitizer	İklendirilmemiş okuma	Clang <code>-fsanitize=memory</code>
Valgrind	Hepsi (yavaş)	<code>valgrind --leak-check=full ./prog</code>
CRT hata ayıklama öbeği	Sızıntı	MSVC <code>_CrtSetDbgFlag</code>

Değerlendirici: işlem bittikten sonra döküm alır, test parolasını arar

Korunan kod bölümlleme: TCB'yi küçült

Güvenilir hesaplama tabanı (TCB): "hatalıysa güvenlik çöker" dediğimiz kodun tamamı

- 200.000 satırlık uygulama, 2.000 satırlık **korunan çekirdek**
- İnceleme, gizleme, bütünlük denetimi yalnız çekirdeğe → maliyet de yalnız orada

Bölümlleme: güvenilir hesaplama tabanını küçült

aralarından yalnız tutamaç ve şifreli veri geçer

GENEL BÖLGE

büyük · hızlı değişir

Arayüz

İş mantığı

hata olasılığı YÜKSEK

KORUNAN ÇEKİRDEK

küçük · seyrek değişir

Dar arayüz (tutamaçlar)

Kripto ve anahtar yönetimi

Bütünlük / RASP denetimleri

incelenebilir, kanıtlanabilir

Güvenlik kritik kod ne kadar küçükse, doğruluğunu kanıtlamak o kadar kolaydır.

Bölümleme seçenekleri

Seçenek	Yalıtım	Örnek
Aynı süreçte ayrı modül	Düşük	Java uygulamasında native kütüphane
Ayrı süreç	Orta	Tarayıcı kum havuzu, yetkili yardımcı süreç (Tarif 1.4)
OS hizmeti	Orta–yüksek	DPAPI, Anahtarlık, Android Keystore
Donanım	Yüksek	Güvenli eleman, TPM, TEE
Sunucuya taşıma	En yüksek	Sır hiç istemciye gitmez, HSM'de kalır

En güçlüsü **sırrı istemciden kaldırmak**; olmazsa donanım; o da yoksa **yazılım koruması** (bu dersin konusu)

Arayüz: dar, doğrulanmış, sırsız

```
typedef struct VaultKey VaultKey;    /* opaque type */

int vault_open(const char *password, size_t n, const unsigned char salt[16],
               VaultKey **out);       /* wipes the password */
int vault_decrypt(VaultKey *a, const unsigned char *cipher, size_t n,
                 unsigned char *plain, size_t capacity, size_t *written);
void vault_close(VaultKey *a);       /* wipes the key */
```

1. **Sırrı dışarı verme, iş yaptır** (tutamaç)
2. Sınırı geçen her şeyi **doğrula**
3. Sınırdan **açık veri** geçirme

Kâhin ve kod sökme

- **Şifre çözme kâhini:** saldırgan `vault_decrypt` 'i kendi verisiyle çağırır — anahtara gerek yok
- **Kod sökme (code lifting):** korunan fonksiyonu olduğu gibi kopyalayıp kara kutu gibi çalıştırır

Önlem	Fikir
Cihaza/örneğe bağlama	Başka ortamda doğru sonuç çıkmaz
Sürüme bağlama	Eski/değiştirilmiş sürümde anahtar çözülmez
Bağlam denetimi	Çağıranın bütünlüğü ve akış sırası doğrulanır
Ortak statik anahtar yok	Bir kopyanın kırılması diğerlerini etkilemez

Şifreleme ile güvenli işleme: açıklık penceresi

Kötü: [aç]  [kapat] anahtar hep açık
İyi: [aç]  [kapat] yalnız işlem anında

1. **Tam zamanında çözme** → fonksiyon dönmeden sil
2. **Örtme**: anahtar XOR rastgele maske — bellekte iki anlamsız parça
3. **Parçalama**: farklı yerlerde, kullanımda birleştir
4. **Kod şifreleme**: fonksiyon kodu çalışma anında çözülür (4, 9. hafta)
5. **Whitebox**: anahtar hiç açılmaz (11. hafta)

Bellekte örtme — kod örneği

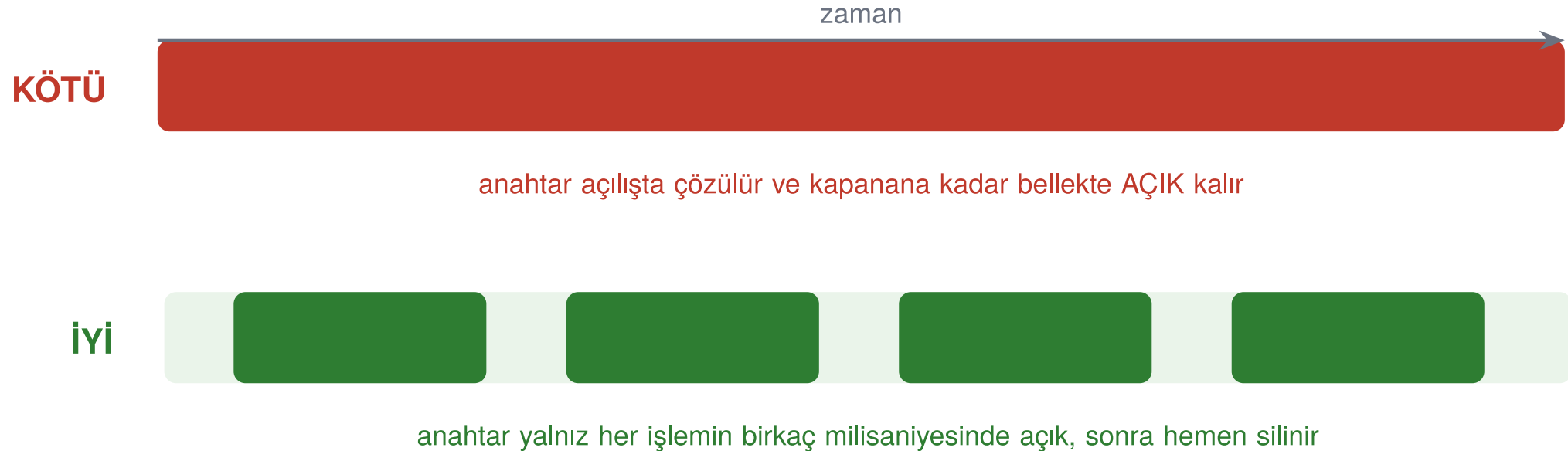
```
typedef struct {  
    uint8_t mask[32];    /* generated randomly at start-up */  
    uint8_t masked[32];  /* key XOR mask */  
} MaskedKey;  
  
static void unmask_key(const MaskedKey *o, uint8_t temp[32])  
{  
    for (size_t i = 0; i < 32; i++)  
        temp[i] = o->masked[i] ^ o->mask[i];  
}
```

Bellekte **anahtarın kendisi değil**, iki anlamsız parça durur; birleştirme yalnız kullanım anında.

Açıklık penceresi — şema

Açıklık penceresi: sır ne kadar süre açık?

aynı işi yapan iki tasarım, çok farklı risk



Bellek dökümü alan saldırgan için önemli olan tek şey: o anda anahtar açık mıydı?

Güvenli geliştirme yaşam döngüsü

Aşama	Etkinlik	Hafta
Gereksinim	Standartlardan güvenlik gereksinimi	13
Tasarım	Tehdit modeli, koruma planı	1–2
Gerçekleştirme	Kodlama kuralları, yasaklı fonksiyonlar, statik analiz	4–5
Doğrulama	Sanitizer, fuzzing, sızma testi	4, 12
Yayın	İmzalı derleme, benzersiz sürüm kimliği	1, 10
Yanıt	Zafiyet bildirimi, acil güncelleme	12

Microsoft SDL · NIST SSDF (SP 800-218)

Benzersiz sürüm kimliği

İncelenen ile dağıtılan aynı mı? → 1.4.2 + git kimliği + binary dosyanın SHA-256'sı

```
execute_process(COMMAND git rev-parse --short=12 HEAD  
                OUTPUT_VARIABLE GIT_COMMIT OUTPUT_STRIP_TRAILING_WHITESPACE)  
target_compile_definitions(kasa PRIVATE VERSION="1.4.2" GIT_COMMIT="${GIT_COMMIT}")
```

```
Get-FileHash .\kasa.exe -Algorithm SHA256 · sha256sum ./kasa
```

Sahada sürüm, **anahtar türetmeye** de katılır → sürüm geri alma saldırısı çalışmaz

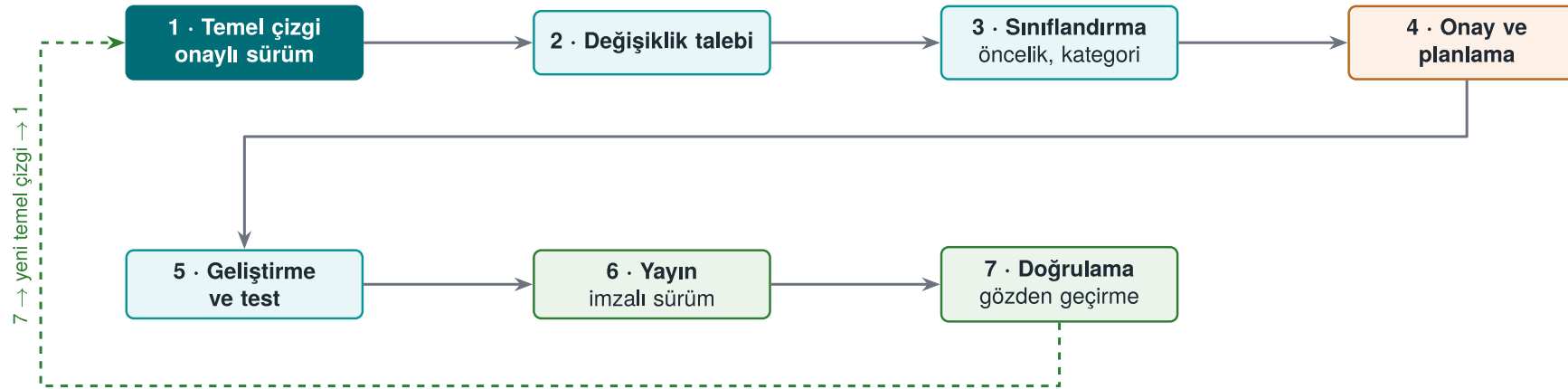
Değişiklik yönetimi: yedi adım

1. Temel çizgi → 2. Değişiklik talebi → 3. Sınıflandırma → 4. Onay ve planlama → 5. Geliştirme ve test → 6. Yayın → 7. Doğrulama
- 3 ile 4 arasında **güvenlik etki analizi**: hangi varlık, arayüz, tehdit, önlem?
 - Yalnız değişen kısmın incelenmesi = **delta değerlendirme** (13. hafta)
 - Depo: korumalı dallar · en az bir gözden geçiren · imzalı etiket · MFA

Değişiklik yönetimi — şema

Değişiklik yönetimi: yedi adım

her değişiklik izlenebilir ve geri alınabilir olmalı



Sertifikasyonda “hangi sürüm onaylandı?” sorusunun cevabı bu döngüdür (12. hafta).

Ödünleşim kaydı

Karar	Seçilen	Gerekçe	Kalan risk
Sürümde günlükleme	Derleme makrosuyla tamamen kaldırıldı	Susturulmuş kod dize olarak kalır	Sahada tanı zor
Hata kodları	Yalnız başarılı/başarısız (opak)	Ayrıntı saldırgana ipucu	Ayrı tanı kanalı
Native "debuggable" bayrağı	Açık (gerekçeli istisna)	Bütünlük denetimi kendi belleğini okumalı	Debugger algılama ile telafi
Parola türetme süresi	250 ms	Bekleme ↔ kaba kuvvet dengesi	Çok zayıf parola

Yazılmamış ödünleşim = değerlendiricinin gözünde **hata**

Bu haftanın araç kutusu

İş	Linux / GCC	Windows / MSVC
Şüpheli kod uyarıları	<code>-Wall -Wextra -Wsign-conversion</code>	<code>/W4</code>
Kütüphanede boyut denetimi	<code>-D_FORTIFY_SOURCE=2 -O2</code>	<code>strcpy_s</code> gibi <code>_s</code> işlevleri
Bellek hatasını yakalama	<code>-fsanitize=address</code>	<code>/fsanitize=address</code>
Bellek dökümü	<code>gdb + gcore</code>	<code>MiniDumpWriteDump</code>
Makine kodu	<code>objdump -d</code>	VS Ayrıştırılmış Kod
Kaldırılmayan silme	<code>explicit_bzero</code>	<code>SecureZeroMemory</code>
Güvenle program çalıştırma	<code>posix_spawn</code> + mutlak yol	<code>CreateProcessW</code> + tam yol

Haftanın araç kutusu — şema

Bu haftanın araç kutusu

hangi araç hangi hatayı yakalar

-Wall -Wextra	şüpheli kod uyarısı	derleme
-Wsign-conversion	işaretili/işaretsiz dönüşüm	derleme
-D_FORTIFY_SOURCE=2	boyutu bilinen tampon denetimi	çalışma
-fsanitize=address	sınır dışı, use-after-free	çalışma
gdb / gcore	bellek dökümü alma	inceleme
strings / objdump -d	ikilideki metin ve kod	inceleme

Derleme zamanı ucuz, çalışma zamanı kesin, inceleme araçları saldırganın da elinde.

8. Proje ve kapanış

Proje: bu hafta yapılacaklar

1. **Takım** (bireysel ya da en çok 4 kişi) ve **konu**
2. GitHub deposu + **proje planı** (iş paketleri, takvim) → onaylat
3. Güvenlik kılavuzu taslağı:
 - **S0** kapak ve sürüm geçmişi · **S2** ürün genel bakışı
 - **S3** mimari ve **arayüz tablosu**
 - **S4** saldırgan modeli · **STRIDE** · **saldırı ağacı**

Sınıf alıştırması (15 dk)

Öğrenci not sistemi: öğretim üyesi not girer, öğrenci görür, veri sunucuda.

1. Veri akış diyagramı + güven sınırları
2. Her STRIDE harfinden **bir** tehdit
3. "Notumu 45'ten 85'e çıkar" saldırı ağacı — **en ucuz yol** hangisi?

Uçtan uca: Kasa'nın savunmasını tamamlamak

Bölüm 4'te "Kasa"nın tehdit tablosunda iki madde vardı; bu haftanın geri kalanı onları **somut kodla** kapatıyor:

Kasa'nın tehdidi	Bu haftaki karşılık
T2 — parola/anahtar bellekte kalır	Bölüm 5: <code>explicit_bzero</code> / <code>SecureZeroMemory</code> , <code>mlock</code> , döküm kapatma (Demo 2)
T8 — ayrıştırıcıda bellek hatası	Bölüm 6–7: sınır denetimi, ASan, güvenli kalıp (Demo 3–4)

Güvenlik bir **his** değil, **süreçtir**: varlığı tanı, tehdidi modelle, katmanla koru, kalan riski yaz. Bu çerçeveyi dönem boyunca kullanacağız.

Klasik hatalar — özet

Hata	Bölüm	Kural
PATH/ortama güvenmek (CWE-426)	5	Mutlak yol + temiz ortam + kabuksuz çalıştırma
Sır bellekte kalır (CWE-14)	5	<code>explicit_bzero</code> / <code>SecureZeroMemory</code> , derleyicinin silemeyeceği biçimde
Arabellek taşması (CWE-121/787)	6	Önce doğrula, sonra sınırlı kopyala; tehlikeli fonksiyonları kullanma
Bir fazla hatası (CWE-193)	6	<code><=</code> değil <code><</code> ; sonlandırıcıya her zaman yer bırak
İşaretli/işaretsiz tamsayı (CWE-195)	6	Boyutları <code>size_t</code> tut; alt ve üst sınırı denetle
Sızıntı, çift serbest bırakma, UAF (CWE-401/415/416)	7	Her bloğun tek sahibi olsun; <code>free</code> sonrası işaretçiyi <code>NULL</code> yap

Altı hatanın hepsi bu destede bir demo ya da kod örneğiyle işlendi; burada tek bakışta toparlıyoruz.

Kendi başına çalış (not verilmez)

1. PATH tuzağı: `ls` (Linux) / `whoami` (Windows) ile tekrarla, tam yolla düzelt
2. Demo 2'de `MOD optimize` → `MOD korumasız`: `memset` şimdi çalışıyor mu?
3. Demo 3'te 16, 17, 18, 19, 20 karakter: `admin` değerleri tablosu
4. `struct session` 'a `balance` ekle: taşmayla 1000'in üstüne çıkar, sonra engelle
5. Kendi projenin STRIDE tablosu (≥ 8 tehdit + önlem)
6. Heartbleed raporunu oku: hangi denetim eksikti? Hangi CWE?

Ayrıntı ve ipuçları: ders sitesi → Hafta 1 → "Kendi başına çalış"

Kendini sına

1. Varlık, tehdit, zafiyet farkı?
2. Alıcı IBAN'ın yolda değişmesi CIA'nın hangisi?
3. Beyaz kutu saldırganı neyi farklı yapar?
4. STRIDE "E" hangi demo?
5. VE / VEYA düğümü — savunma için hangisi iyi?
6. `system("date")` neden tehlikeli? Üç düzeltme?
7. `memset` neden kayboldu?
8. ASan 17 karakteri neden yakalamadı?
9. `-1` `memcpy` 'ye gidince ne olur?
10. Derinlemesine savunma — mobil ödeme mimarisinden örnek?

Kendini sına — cevaplar (1–10)

1. **Varlık** korunacak değerli şey; **tehdit** zarar verebilecek olası olay/aktör; **zafiyet** tehdidin kullanabileceği kusur.
2. **Bütünlük (Integrity)** — yoldaki IBAN'ın değişmesi veri bütünlüğü ihlalidir.
3. **Beyaz kutu (MATE)** programa **sahiptir** (belleği/anahtarı okur, kodu değiştirir); ağ saldırganı yalnız **dışarıdan** bağlanır.
4. **E = Elevation of Privilege** (yetki yükseltme) → ayrıcalık bırakma / `setuid` demosu.
5. **VE (AND)** savunmaya iyi: hedef tüm alt adımları ister, **birini** kesmek saldırıyı durdurur. VEYA saldırgana kolaylık.
6. `system` **kabuğu** yorumlar → komut enjeksiyonu/PATH ele geçirme. Düzelt: `execvp` ile **doğrudan** çağır (kabuksuz), **tam yol**, girdi allowlist.
7. Derleyici **ölü mağaza** olarak sildi (sonra okunmuyor). `memset_s` / `explicit_bzero` / `volatile` kullan.
8. Taşma **redzone**'a ulaşmadı / o kod yolu çalışmadı; ASan yalnız **enstrümante** bölgeyi görür.
9. `-1`, `size_t` 'te **SIZE_MAX**'e döner → devasa kopya → taşma/çökme.
10. Ödeme: TLS+pinning (aktarım) · AEAD (bekleme) · RASP/anti-debug · whitebox/HSM anahtar · sunucu denetimi — **katman**.

Kendini sinama (devam)

1. Bir veri akışına neden **S** sorulmaz?
2. Tehdide dört yanıt nedir? "Ortadan kaldırma"ya örnek verin
3. setuid programda neden **önce grup** bırakılır?
4. `strncpy` neden güvenli `strcpy` değildir?
5. Heartbleed neden hiçbir şey çökmeden veri sızdırdı?

Kendini sinama — cevaplar (devam 1–5)

1. Veri akışına **S sorulmaz**: akışın kimliği yoktur (sahtecilik uçlara/varlıklara sorulur).
2. **Azalt · kaldır · aktar · kabul et**. "Kaldırma" örneği: "parolayı hatırla" özelliğini **hiç sunmamak**.
3. `setuid` 'de **önce grup** bırakılır; kullanıcı önce bırakılırsa grubu değiştirecek **yetki kalmaz**.
4. `strncpy` uzun kaynakta sonuna sonlandırıcı **koymaz** → sonlandırılmamış dize.
5. Heartbleed yalnız **okudu**; okunan bellek sürecin kendi **geçerli** belleğiymiş → çökmeden sızdı.

Kendini sinama (devam)

6. Kanarya taşmayı fark edince ne yapar? Neden hâlâ sorun?
7. `free(p); p = NULL;` neyi önler, neyi önlemez?
8. `mlock` neyi önler, neyi önlemez?
9. "Şifre çözme kâhini" nedir? Bir karşı önlem?
10. Benzersiz sürüm kimliğinin üç bileşeni?

Kendini sinama — cevaplar (devam 6–10)

6. Kanarya bozulunca programı **sonlandırır** (hizmet reddi); bitişik değişkene taşmayı **görmez**.
7. `free(p); p=NULL;` **çift bırakma/UAF**'ı önler; **takma adları** (aynı adrese başka işaretçi) korumaz.
8. `mlock` belleğin **takasa** yazılmasını önler; **okunmasını** ve silinmemiş sırrı önlemez.
9. **Şifre çözme kâhini**: anahtarsız çözdürme imkânı. Karşı önlem: bağlam/bütünlük denetimi, çağrı sınırı, **cihaza bağlama**.
10. Benzersiz sürüm: **anlamsal sürüm + git kimliği + binary SHA-256**.

Sözlük

Terim	Anlam
CIA	Gizlilik/bütünlük/erişilebilirlik
MATE	Programa sahip saldırgan
STRIDE	Altı tehdit sınıfı
Saldırı ağacı	Hedefi alt adımlara bölme
DFD	Veri akış diyagramı
Katmanlı savunma	Çok önlem birlikte

Bir sonraki hafta

2. hafta — Bilgisayar virüsleri ve güvenlik modelleri

Bu hafta kurduğumuz tehdit modelleme (STRIDE, saldırı ağacı) ve saldırgan modeli araçlarını, gelecek hafta somut bir tehdit sınıfına — zararlı yazılıma — uygulayacağız: bir virüsün nasıl yayıldığını, nasıl gizlendiğini ve hangi erişim denetimi modelleriyle (DAC/MAC/RBAC) durdurulduğunu göreceğiz.

Bu haftaki bellek güvenliği hataları için gördüğümüz CWE sınıflandırması, 2. haftada CVE ve CVSS ile birlikte derinleşecek.